

EITSIM Studio

Real-Time EIT Patient Simulator — Technical Overview

Ilan Jordan

Timpel Medical

This is an abridged, public version of the system's internal design document, prepared for portfolio use. Sections covering device wire protocols, transport credentials, internal document references, patient-derived scenario data and internal verification findings have been removed. It is shared to show the scope and style of the engineering documentation, not as a specification.

Objective and Scope

Purpose

EITSIM Studio (Electrical Impedance Tomography Simulator Studio) is a Hardware-in-the-Loop (HIL) bench instrument that simulates the respiratory mechanics and hemodynamics of a mechanically-ventilated ICU patient. Its purpose is to drive and verify Electrical Impedance Tomography (EIT) medical devices — the EIT Simulator, the EITmini, and a real, unmodified Enlight PIM (Processing & Interface Module) — with a physiologically realistic, fully controllable, and perfectly repeatable signal source, so that device behaviour can be exercised and validated without a patient and without a physical phantom.

The application couples a validated C++ numerical engine (eitsim_core) to a PySide6/Qt6 graphical front end styled as an ICU monitor. The engine integrates a non-linear multi-compartment lung model together with cardiac pulsatility and regional perfusion; the front end lets an operator configure the patient and the ventilator, observe the resulting waveforms globally and by lung region, perform respiratory maneuvers, and route the simulated physiology out to the connected EIT hardware.

Capabilities

EITSIM Studio allows a user to:

- Select from a library of predefined, clinically-derived patient scenarios, and import/create custom scenarios.
- Configure and apply the mechanical ventilation modes VCV, PCV, Spontaneous and HFOV, including superposition of spontaneous breathing with HFOV to simulate new treatments.
- Visualise real-time pressure, flow and volume waveforms both globally and per lung compartment (Anterior, Posterior, Right, Left), together with the ECG and regional hemodynamic signals.
- Perform and analyse respiratory-mechanics maneuvers, including the static pressure–volume (P–V) curve, PEEP titration, the Costa collapse/overdistension method, and a pressure-ramp maneuver.
- Operate as a digital twin under Hardware-in-the-Loop control of a real Hamilton C6 ventilator, whose live airway pressure and flow drive the simulated lung in place of the internal ventilator.
- Emulate the EIT Acquisition Module (AM) so that a real, unmodified PIM performs live EIT monitoring driven entirely by the simulated physics.
- Record continuous raw data to CSV and save/load the complete application state.

Intended audience

The document and the application address three operator personas defined in the software requirements:

- Operator — a clinically-oriented user who works with global ventilation metrics and the ICU-monitor presentation.
- Researcher — a user who edits the underlying physiological (NL_MC) model parameters to study regional behaviors.
- Test Engineer — a user who exercises the metrological and Hardware-in-the-Loop functions: packet-loss and link-quality checks, CSV capture, and device bring-up.

Scope of this document

This document is a complete, as-built technical description of the EITSIM Studio application: its architecture, its physics engine and patient model, its ventilation modes and analysis suite, its user interface, its four hardware interfaces, its data-recording and build/deployment characteristics, and the verification evidence gathered during development. The three-month development history is recorded factually within the relevant

sections rather than as a separate narrative.

Content that is fully specified in companion controlled documents is summarized and referenced here rather than reproduced. In particular, the AM↔PIM wire protocol is specified in [internal ref] , the EIT Simulator control interface in [internal ref], and the detailed numerical derivations of the lung model in the associated design/mathematics documents. The reader is directed to those documents for the underlying detail; see the References section.

Revision note

This is Revision 1 of [internal ref]. It is a comprehensive expansion of Revision 0 ([internal ref], "EITSim Studio"), covering the full system as delivered — including the Hamilton C6 Hardware-in-the-Loop capability and the AM/PIM emulator, which post-date the original release.

System Overview

What EITSIM Studio is

EITSIM Studio is a desktop application that presents itself as an ICU monitor and behaves as a complete, self-contained simulator of a ventilated patient's thorax (heart and lungs). A physics engine advances a model of the lungs and cardiovascular system in real time; the graphical layer renders the resulting signals and lets the user drive the simulation; and a set of hardware interfaces routes the simulated physiology to, or accepts live control from, external EIT and ventilation devices.

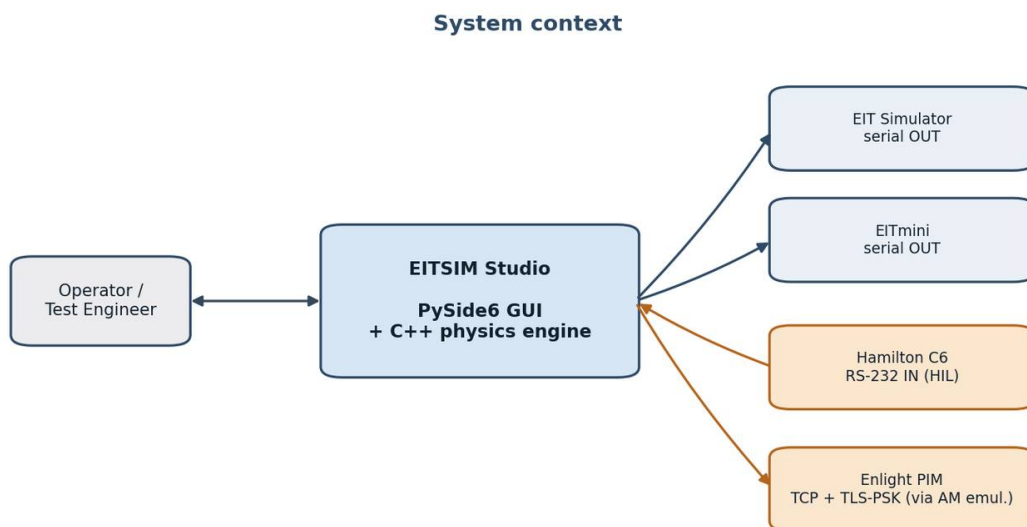


Figure 1. Top-level context: the operator, EITSIM Studio, and the four hardware interfaces.

Because the physiology is generated by a deterministic model rather than captured from a patient, every test is repeatable and every parameter is controllable. This is what makes the application useful for device verification: an EIT device can be presented with a precisely known, reproducible respiratory and cardiac signal, and its response compared against ground truth.

From the legacy application to the current architecture

The current application is a re-implementation of an earlier C++ desktop tool built on the FLTK toolkit with MathGL plotting. The numerical core of that tool — the lung model, the ventilation-signal generator, the

estimators and the analysis routines — was mature and validated, so it was retained. The graphical application around it was rebuilt in Python 3 using PySide6 (Qt 6) for the interface and pyqtgraph for the real-time scopes.



Figure 2. The legacy FLTK/MathGL C++ application that preceded the PySide6 rewrite.

The retained C++ engine is exposed to Python as a compiled extension, `eitsim_core`, through a `pybind11` binding. The Python layer therefore owns all presentation, interaction, state management and device I/O orchestration, while the C++ layer owns the numerics. This separation keeps the validated physics untouched while allowing the user experience and the hardware integrations to evolve rapidly in Python.



Figure 3. EITSIM Studio running on the bench touchscreen (Operation view).

What is new in this version

EITSIM Studio is a complete re-engineering of the earlier FLTK/MathGL desktop tool. The validated C++ numerical engine was retained (now exposed through pybind11), while the entire application around it — the user experience, the instrumentation and every hardware integration — is new. The most significant additions relative to the legacy version are summarised below.

Table 1. Legacy FLTK version compared with the current EITSIM Studio.

Area	Legacy FLTK version	Current EITSIM Studio
UI toolkit	C++ with FLTK and MathGL plotting	Python 3 with PySide6 (Qt 6) and pyqtgraph
Numerical engine	Compiled in-process (C++)	The same validated engine, retained and exposed via pybind11 (eitsim_core)
Presentation	Desktop windows and forms	ICU-monitor experience: permanent numeric dashboard, sweeping scopes, bottom navigation, full-screen
Regional disclosure	Compartment waveforms	Dedicated Disclosure view — per-compartment volume/flow/alveolar pressure plus ECG and regional hemodynamics, colour-coded (US37)
Hamilton C6 connectivity	None	NEW — Hardware-in-the-Loop: a real Hamilton C6 drives the simulated lung over RS-232 (Block Protocol)
EIT PIM connectivity	Regional conductivities to the EIT Simulator only	NEW — full AM emulation over TLS 1.3-PSK so a real, unmodified PIM runs live EIT
On-screen guidance (Manual)	None	NEW — a Manual mode: tap or hover any control for an inline explanation
Event Log	None	NEW — a timestamped, colour-coded Event Log of lifecycle, hardware and error events
Data recording	Limited	Two CSV backends (Python 50 Hz / Engine 100 Hz) and the built-in plot_log viewer
State persistence	Limited	Full application-state save/load to .eitsim; auto-saved session settings
System health	None	Merged link-drop-rate and engine-error health badge with HIL acceptance thresholds
Analysis	P–V and titration plots	Mode-differentiated PEEP titration, P–V curves and the Costa method with CSV export and reference overlay
Touch / cross-platform	Desktop	Touch-friendly inputs; Ubuntu 24.04 / 20.04 / Windows with per-ABI engine binaries

The four hardware interfaces at a glance

EITSIM Studio connects to external equipment through four distinct interfaces, described in full in the Hardware Interfaces chapter:

Table 2. Summary of the four hardware interfaces.

Interface	Direction	Transport	Role
EIT Simulator	Output (Studio → device)	Serial	Streams the simulated regional signal to the EIT Simulator; also provides a precise 100 Hz hardware clock.
EITmini	Output (Studio → device)	Serial	Streams ventilation data to the EITmini device.
Hamilton C6	Input (device → Studio)	Serial RS-232	A real ICU ventilator drives the simulated lung (Hardware-in-the-Loop), replacing the internal ventilator.

Interface	Direction	Transport	Role
AM / PIM emulator	Output (Studio → PIM)	TCP + TLS 1.3-PSK	Studio emulates the EIT Acquisition Module so a real, unmodified PIM runs live EIT from the simulation.

Core concepts

Three concepts recur throughout this document and are introduced here:

- Regional (4-compartment) model. The lung is modelled as four compartments — Anterior, Posterior, Right and Left — with a fifth Heart compartment for hemodynamics. A fixed colour convention is used everywhere in the interface: Anterior = Yellow, Posterior = Blue, Right = Green, Left = Red.
- Two clocks. The physics engine advances at a fixed 100 Hz, while the EIT/AM acquisition runs at 50 frames per second and the Hamilton C6 streams at 50 Hz; rate conversions (for example a zero-order interpolation - hold - from 50 Hz to 100 Hz) are applied at the boundaries.
- ICU-monitor user experience. The interface deliberately imitates a bedside monitor: a permanent left-hand numeric dashboard, sweeping waveform scopes, a bottom navigation bar, and a system-health indicator.

How to read this document

The Software Architecture chapter describes the layering and threading model. The Physics Engine, Patient Model, Ventilation Modes and Analysis chapters describe what the simulator computes. The User Interface and Live Measurements chapters describe what the operator sees and does. The Hardware Interfaces chapter describes the four external connections, including the Hamilton C6 HIL and the AM/PIM emulator. Build/deployment, verification, limitations, references and appendices follow.

Software Architecture

Architectural pattern

EITSIM Studio follows a reactive Model-View-Controller (MVC) pattern that bridges the PySide6 user interface to the legacy C++ engine. The Model is the C++ numerical engine (`eitsim_core`), which is blind to all input and output. The View is the collection of Qt views and widgets under `src/ui`. The Controller is `src/core/app_state.py`, whose `AppState` object is the single source of truth: it wraps exactly one `SimulationFacade`, mediates every interaction between the UI and the engine, and performs unit conversion at the boundary.

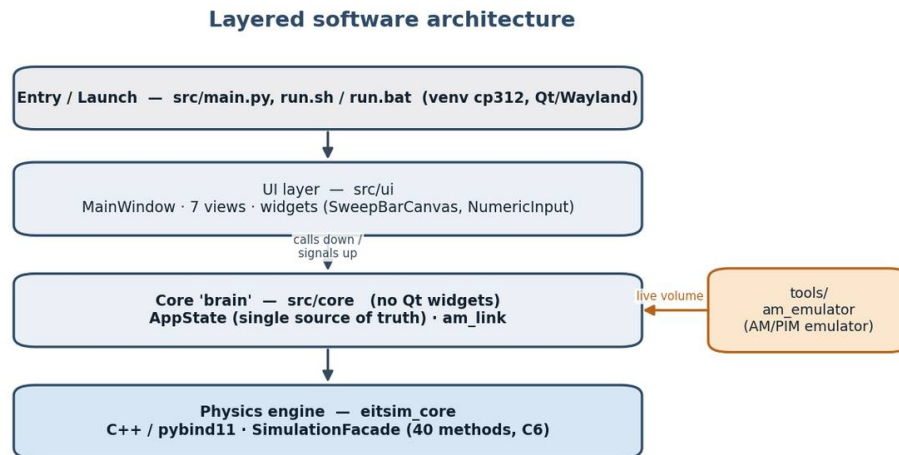


Figure 4. Layered software architecture.

Layers

The codebase is organised into four layers plus an out-of-process tool:

- Entry / launch — src/main.py, launched via run.sh (Linux/WSLg) or run.bat (Windows). It configures the engine import path, application fonts, optional OpenGL-accelerated scopes, and a custom event-driven tooltip system, then constructs AppState and the main window.
- UI layer — src/ui, containing main_window.py (navigation and a QStackedWidget of views), one view per tab under views/, and a reusable widget library under widgets/.
- Core 'brain' — src/core, containing no Qt widgets. AppState is the single source of truth; am_link.py is the embedded AM/PIM emulator controller; platform_env.py centralises operating-system branching.
- Physics engine — eitsim_core, the prebuilt pybind11 extension exposing the SimulationFacade class.

The AM/PIM emulator itself lives under tools/am_emulator and is engine-agnostic; the core layer runs it in-process through am_link.py.

The AppState controller

AppState (a QObject) constructs one SimulationFacade and registers its chunk callback with a chunk size of five samples; because the engine ticks at 100 Hz, the callback fires roughly every 50 ms with five samples. AppState exposes the engine lifecycle (start/stop), the ventilation-mode and maneuver setters, the physiology setters, the static-analysis calls and the hardware-connection calls. It performs all unit conversion at the boundary (for example breaths-per-minute to Hz, millilitres to litres, and inspiratory-to-expiratory ratio inversion). It publishes data to the rest of the application through Qt signals, including paced_sample, metrics_updated, model_error_changed, ramp_finished and breath_completed.

The pybind11 boundary and callback pacing

The engine delivers data through an asynchronous chunk callback registered with register_ui_callback(callback, chunk_size). The callback, AppState._on_engine_chunk, is invoked by the C++ physics thread. Because the callback receives a reference to a C++ buffer that is cleared and reused immediately after the callback returns, the callback extracts every scalar and list into plain Python objects

before doing anything else, and it never touches Qt widgets — it only rebinds plain values and emits signals. Samples are appended to a FIFO queue and released to the interface on a wall-clock schedule.

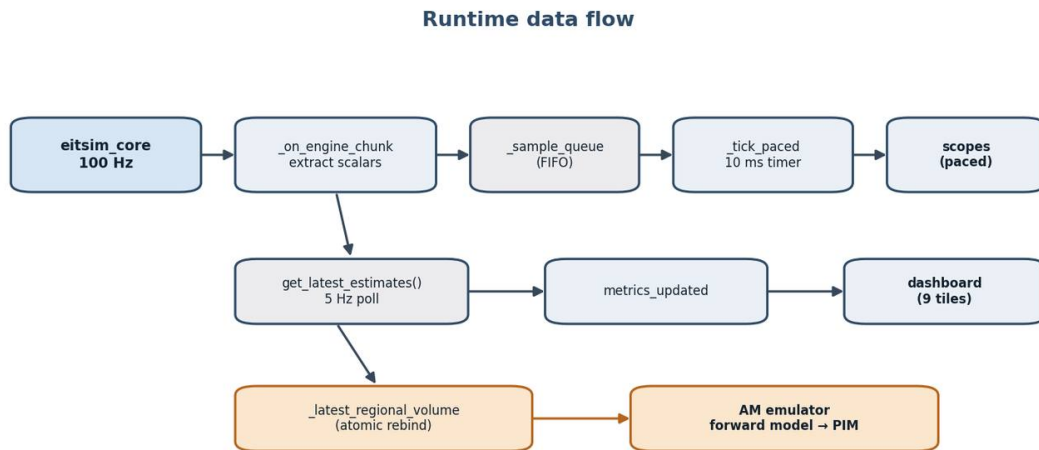


Figure 5. Runtime data flow from the 100 Hz engine to the paced scopes and the 5 Hz dashboard.

A Qt.PreciseTimer running at 10 ms (**_tick_paced**) drains the sample queue and emits exactly one **paced_sample** per real engine frame, re-anchoring to the wall-clock target when the queue runs dry. A separate 200 ms (5 Hz) timer polls the engine for the latest respiratory-mechanics estimates and emits **metrics_updated** for the numeric dashboard.

Threading model

Three thread contexts interact, and this is the most defect-prone part of the system. Correctness depends on strict discipline about which thread touches which data.

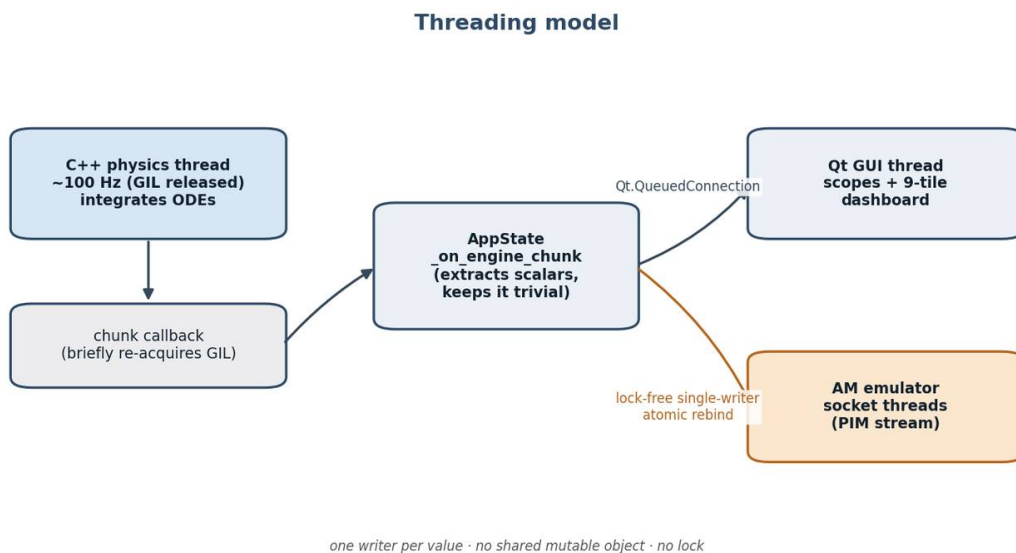


Figure 6. Threading model: the C++ physics thread, the Qt GUI thread, and the AM-emulator socket threads.

- C++ physics thread — integrates the model at ~100 Hz with the GIL released, briefly re-acquiring the GIL to invoke the chunk callback. The callback is kept trivial.
- Qt GUI thread — receives all per-sample data and metrics exclusively through Qt.QueuedConnection signals, so UI slots always run on the GUI thread.
- AM-emulator socket threads — serve the PIM connection and read the latest simulated regional volume.

The cross-thread publishing rule is a lock-free single-writer atomic rebind: a value shared across threads (for example `self._latest_regional_volume = [...]`) is published by rebinding a plain scalar or list, with a single writer and no lock. A mutable object is never shared for two threads to mutate. This pattern — together with keeping the C++ callback trivial, routing all GUI updates through Qt.QueuedConnection, and guarding hot-loop frame builders so a bad frame is skipped rather than fatal — is the safe idiom the codebase relies on.

Packaging and ABI

The engine is shipped as prebuilt per-ABI binaries. `main.py` places `src/core` and then `src/Library_cpp` on the import path and lets CPython select the binary whose ABI tag matches the running interpreter (the reference environment is the `eitsim_env` virtual environment, CPython 3.12). Build and deployment are described in the Build, Deployment and Runtime chapter.

Physics Engine (`eitsim_core`)

The `eitsim_core` extension

`eitsim_core` is a prebuilt C++ extension, bound to Python with `pybind11`, that exposes a single public class: `SimulationFacade`. The current build exposes 40 public methods, including the Hamilton C6 receive-link API (`connect_c6_ventilator`, `get_c6_status`, `disconnect_c6_ventilator`); the presence of these methods is the sanity gate for a correct build. The facade is the only surface the Python layer uses; all model internals are private to the C++ side.

Callback and chunk system

The engine core runs at a fixed 100 Hz. A user-interface callback is registered with `register_ui_callback(callback, chunk_size)`; with a chunk size of five, the callback fires every 50 ms carrying five `SimulationOutput` samples. Each `SimulationOutput` contains a `respiratory_state` (`SimulationState`) and a `hemo_output` (`HemodynamicOutput`).

`SimulationState` carries the total time, the global volume and flow, the last airway-pressure sample, the four per-compartment volumes, flows and elastic pressures (index order 0 = Anterior, 1 = Posterior, 2 = Right, 3 = Left), the ventilation phase, and the last model error code. `HemodynamicOutput` carries the ECG signal and the per-region pulsatility values. The full structure and method reference is given in Appendix B.

Physics libraries

The engine is composed of several numerical libraries, each with a single responsibility:

Table 3. Physics and estimation libraries in the engine.

Library	Responsibility
<code>nl_mc_model</code>	Non-linear multi-compartment lung model (four compartments). Uses the Rohrer parabolic pressure–flow relation $\Delta P = R_0 \cdot f + R_1 \cdot f \cdot f $ and the Venegas sigmoidal pressure–volume compliance curve, integrated step-by-step over the ventilator pressure/flow samples. It is the source of the model error codes.

Library	Responsibility
ventsimulator	Decoupled, time-domain ventilator signal generator. Produces the control intent (pressure or flow) for VCV, PCV, Spontaneous and HFOV modes.
hemodynamic_model	Cardiogenic phenomena: cardiac pulsatility and saline-bolus (contrast) perfusion, using a Fourier and time-domain formulation.
LSFestimator	Real-time, breath-by-breath respiratory-mechanics estimator (plateau pressure, static compliance, inspiratory/expiratory resistance, AutoPEEP) computed by least-squares fitting of the pressure/flow/volume waveforms.
pv_analysis	Offline static pressure–volume maneuver analysis: compliance, collapse and hyperdistension curves from a deflation ramp (the Costa method).
patient_library	Self-contained predefined patient scenarios (lung mechanics plus default ventilator settings), validated against real patient data.

Numerical methods

The lung ordinary-differential-equation system is integrated with an adaptive-step Runge–Kutta Dormand–Prince (Dopri5) solver (Boost.odeint), with a step-limiter observer for stability. The non-linear Rohrer resistance and the sigmoidal Venegas compliance make each step non-linear; junction-pressure root finding uses the TOMS 748 algorithm (Boost.math). The detailed derivations are given in the engine design/mathematics documents and are not reproduced here; see the References section and patient_model_parameters.md.

Model error codes

Every simulation sample carries a model error code. AppState maps these to human-readable text and reports transitions rather than repeating; the System Health indicator escalates them. Critical codes take priority over link status.

Table 4. Engine model error codes.

Code	Meaning	Severity
0	Success — no error	—
-1	Invalid solver handle	Critical
-2	Invalid setter argument	Critical
-3	Invalid patient parameters	Warning
-4	Runge–Kutta solver failure	Critical
-5	Solver step-limiter tripped / instability	Warning
-6	Volume exceeded total lung capacity (barotrauma)	Critical

Two clocks

The engine core advances at a fixed 100 Hz. The EIT Simulator hardware, when connected, provides a precise 100 Hz clock that paces acquisition; the AM/PIM EIT acquisition runs at 50 frames per second; the Hamilton C6 streams at 50 Hz and is upsampled to the engine's 100 Hz by a zero-order hold. These rate relationships recur in the hardware chapters.

Patient Model and Scenarios

Structure of the patient model

A patient in EITSIM Studio is defined by three coupled subsystems: the ventilation response (the lung mechanics), the cardiac pulsatility, and the regional perfusion. Default values for all three come from the

engine's patient_library, whose scenarios were tuned against real patient data. Parameters are edited in the Patient view and applied atomically; the full parameter dictionary is given in patient_model_parameters.md and summarised here.

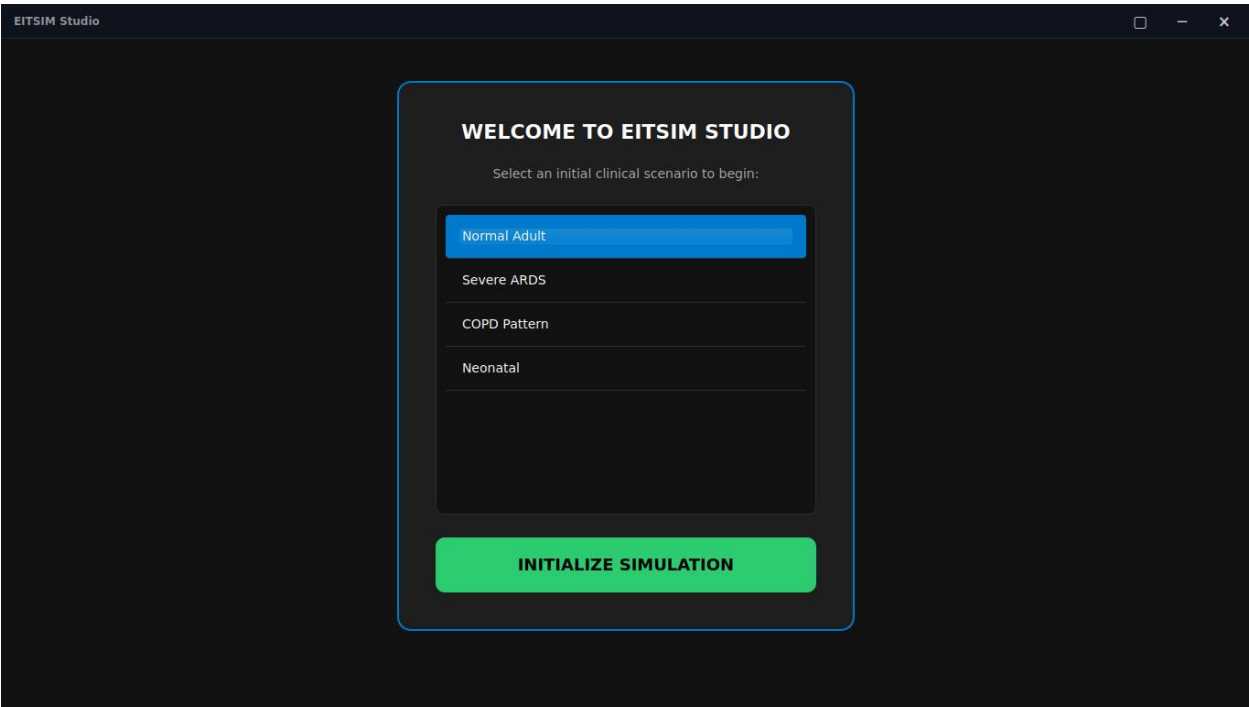


Figure 7. Startup scenario selector — choosing an initial clinical scenario.

Ventilation model (NL_MC)

The lung is a non-linear multi-compartment model with four compartments — Anterior, Posterior, Right and Left. Global airway parameters describe the shared circuit: R0_circuit (linear resistance) and R1_circuit (the quadratic Rohrer term). Each compartment then carries its own mechanics.

Table 5. Per-compartment lung-mechanics parameters.

Parameter	Meaning	Unit
R0	Linear (Rohrer) airway resistance	cmH2O/(L/s)
R1	Quadratic (Rohrer) airway resistance	cmH2O/(L/s)^2
Cmax	Maximum compliance	mL/cmH2O
VR	Residual volume	mL
VTLC	Total lung capacity	mL
PmaxC	Pressure of maximum compliance (sigmoid inflection)	cmH2O
V_initial	Optional initial volume	mL

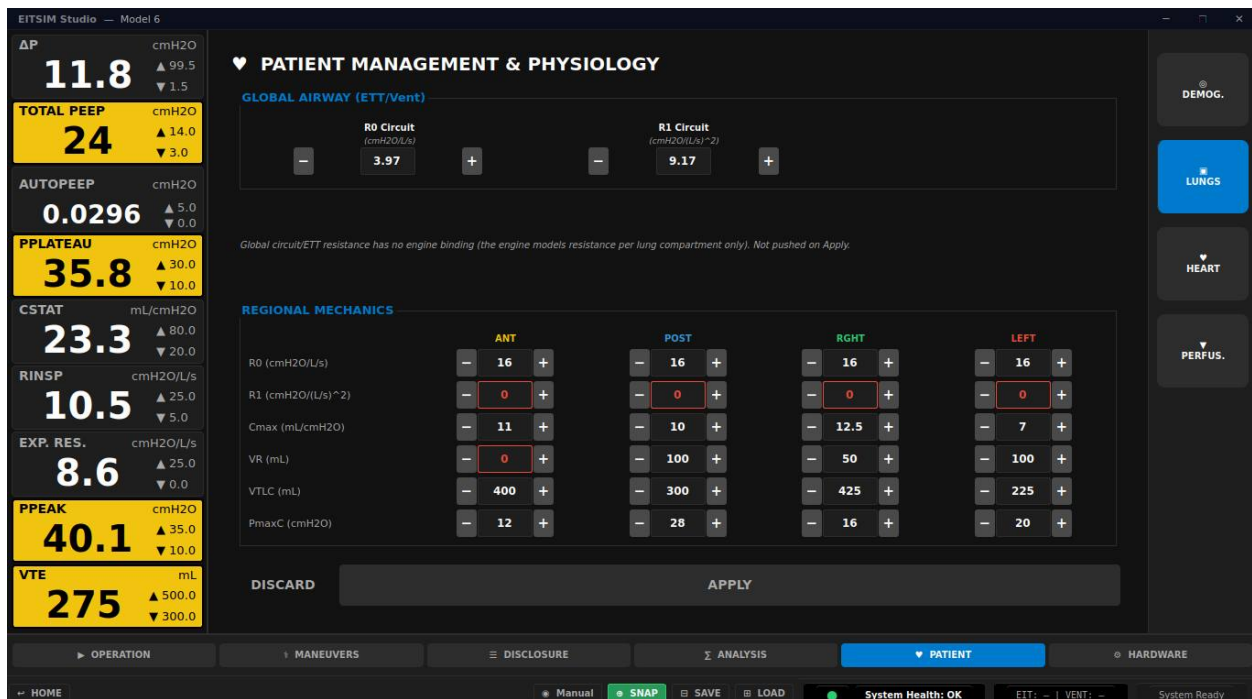


Figure 8. Patient view — global airway and per-compartment respiratory mechanics.

Pulsatility model

The pulsatility model adds a fifth compartment, the Heart, and produces the cardiogenic impedance signal and an ECG reference. Global pulsatility parameters are the heart rate, the heart-rate-variability (HRV) amplitude and frequency, and a base-waveform identifier that selects a predefined heart-and-pulmonary waveform set derived from real data. The generated ECG is a simple trapezoidal trigger train — sufficient to trigger QRS detection — and is not a realistic P/QRS/T morphology. Per region (Anterior, Posterior, Right, Left, Heart) the model carries a gain (impedance impact, expressed in millilitre-equivalent air) and a phase in degrees relative to the QRS.

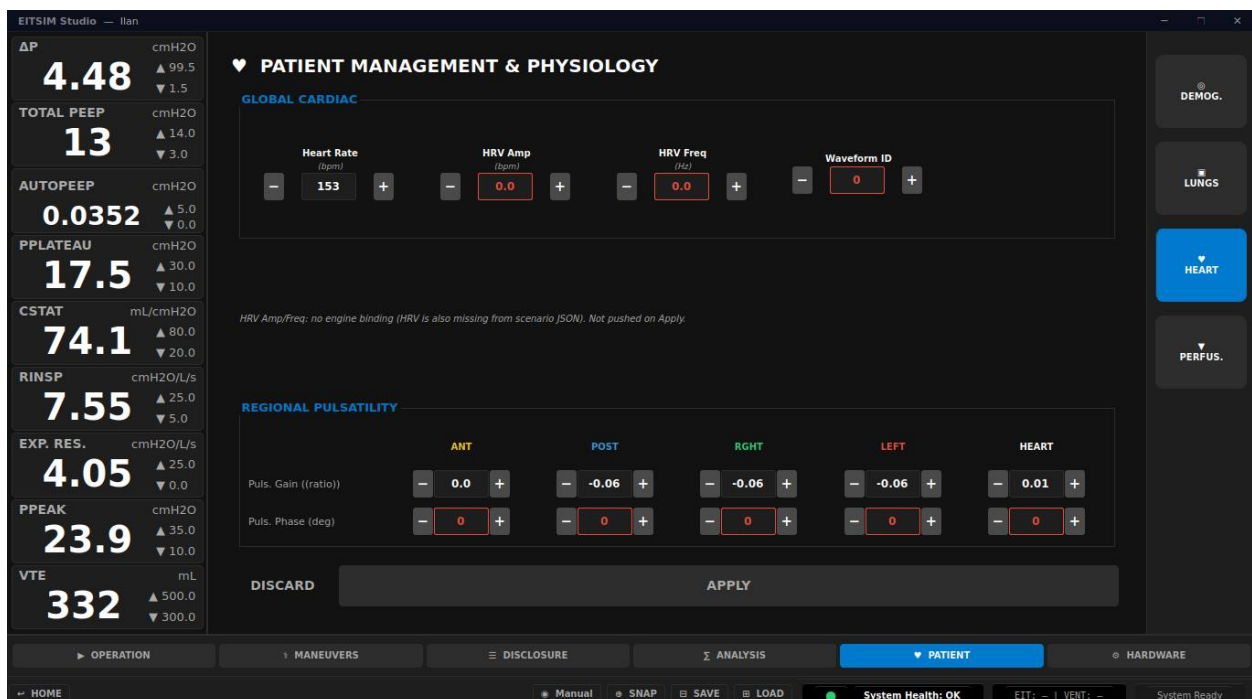


Figure 9. Patient view — cardiac rate/HRV and regional pulsatility parameters.

Regional perfusion (contrast bolus)

The perfusion model represents a saline (contrast) bolus as a gamma-variate curve per region (Anterior, Posterior, Right, Left, Heart). Each region is described by an appearance time, a time-to-peak, a mean transit time, and a peak value (impedance change in millilitre-equivalent air).

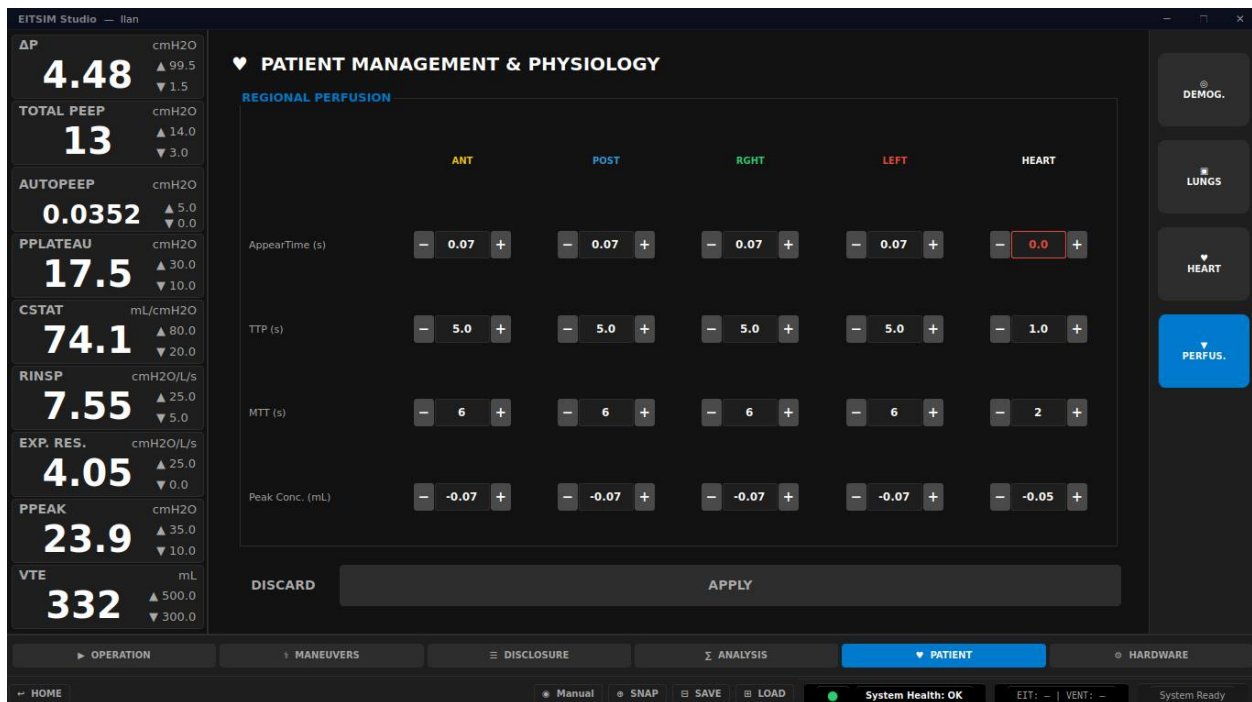


Figure 10. Patient view — regional perfusion (contrast-bolus) parameters.

Demographics

Each patient also carries demographic fields — name, identifier, clinical note, age, sex, weight and height — shown in the Patient view. These are descriptive; the physiological behaviour is governed by the mechanics, pulsatility and perfusion parameters above.

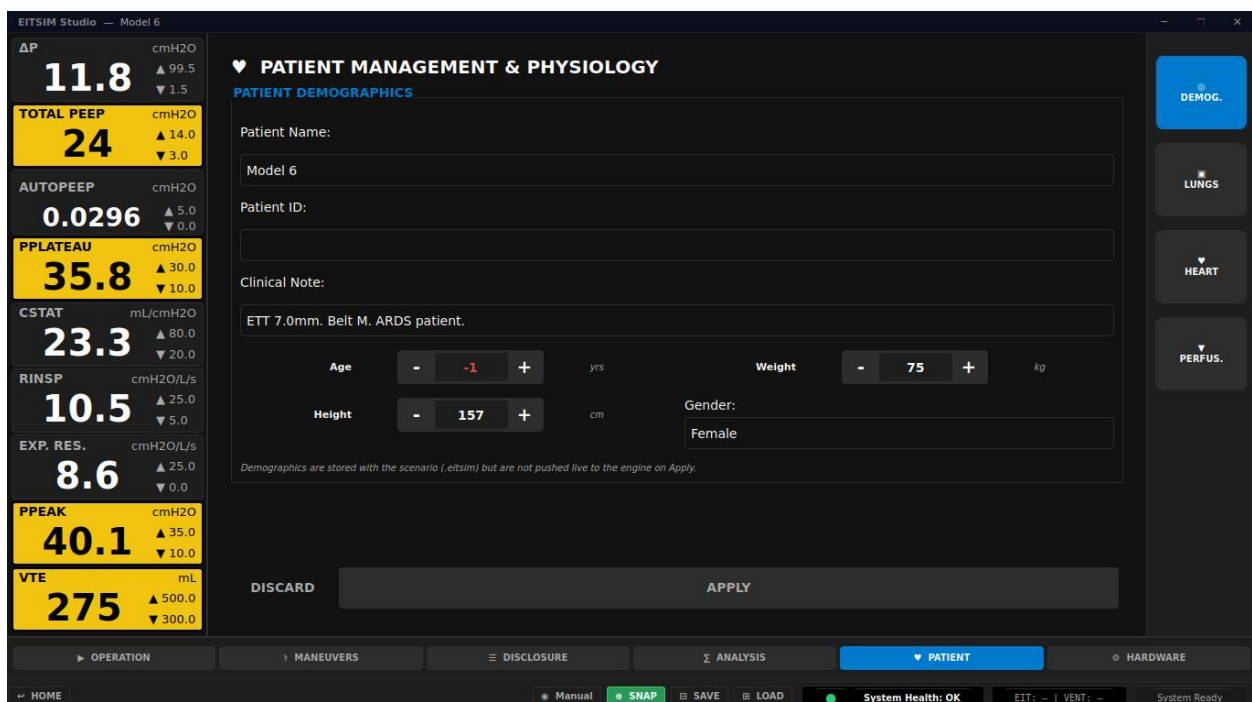


Figure 11. Patient view — demographics and clinical note.

Custom scenarios

Custom scenarios can be imported; they are stored under `~/eitsim_studio_scenarios/`, tagged (custom) in the selector, and accept either an engine scenario JSON or a saved .eitsim state file.

Scenario validation

The library scenarios were validated against real, anonymised PEEP-titration patient data used as ground truth. Model parameters (R_0 , R_1 , VR , $VTLC$, P_{maxC} , C_{max}) were tuned iteratively and the model's global compliance versus PEEP, the Anterior/Right regional ratios versus PEEP, and the Costa collapse/hyperdistension curves versus PEEP were compared against the reference data. See the Verification and Validation chapter.

Ventilation Modes and Maneuvers

Overview

EITSIM Studio provides four ventilation modes — Volume-Controlled (VCV), Pressure-Controlled (PCV), Spontaneous and High-Frequency Oscillatory (HFOV) — plus a pressure-ramp maneuver and a saline-bolus maneuver. The ventilator behaviour is generated by the engine's ventsimulator, which produces a control intent (either a target pressure or a target flow) that drives the lung model. The parameters and waveform shapes summarised here follow `ventilation_modes.md`; that document is the authoritative reference for the waveform mathematics.

A unit caveat applies throughout: the engine library expresses frequency in Hz, while the interface uses breaths per minute. AppState performs the boundary conversions — breaths-per-minute to Hz (divide by 60), millilitres to litres (divide by 1000), and inspiratory-to-expiratory ratio inversion for the pressure-controlled modes.

Volume-Controlled Ventilation (VCV)

VCV is parameterised by frequency, inspiratory time (T_{insp}), inspiratory pause, inspiratory flow and PEEP. The breath is generated in three phases: a flow-controlled inspiration at the set flow, a flow-controlled pause at zero flow, and a pressure-controlled expiration held at PEEP.



Figure 12. Operation view — Volume-Controlled Ventilation (VCV).

Pressure-Controlled Ventilation (PCV)

PCV is parameterised by frequency, inspiratory-to-expiratory ratio, PEEP and the driving pressure ΔP . The cycle time is $T_{\text{cycle}} = 60/\text{frequency}$ and the inspiratory time is $T_{\text{insp}} = T_{\text{cycle}}/(1 + 1/\text{IE})$; the inspiratory output is held at $\text{PEEP} + \Delta P$.



Figure 13. Operation view — Pressure-Controlled Ventilation (PCV).

Spontaneous mode

Spontaneous breathing is parameterised by frequency, the inspiratory rise, hold and release fractions of the cycle, and the peak negative muscular pressure (Pmus). It is always pressure-controlled, with a quarter-sine rise, a hold, and a quarter-cosine release.



Figure 14. Operation view — Spontaneous mode: global volume, flow and airway-pressure sweep scopes.

High-Frequency Oscillatory Ventilation (HFOV)

HFOV is parameterised by frequency (in Hz), inspiratory-to-expiratory ratio, mean airway pressure and ΔP , with a selectable sine or square waveform. For the sine variant the inspiratory and expiratory half-cycle frequencies are set so the integral averages to the mean pressure; the square variant shifts the amplitudes according to the I:E ratio.



Figure 15. Operation view — High-Frequency Oscillatory Ventilation (HFOV) – Sine wave.

Superposition

Spontaneous effort combined with HFOV, or a PEEP level combined with a saline bolus. Superposition is offered as a toggle within the Operation panel under HFOV, rather than as a separate mode, or toggled by simply selecting a maneuver while running the desired ventilatory mode.



Figure 16. Operation view — High-Frequency Oscillatory Ventilation (HFOV) – Square wave + Spontaneous Breathing.

Pressure-ramp maneuver

The pressure-ramp maneuver produces a slow inflation/deflation used to measure the quasi-static pressure–volume relationship. It is parameterised by the start pressure (P_{start}), the top pressure (P_{top}), the end PEEP, the ramp speed (in cmH₂O/s) and a top pause (T_{pause}). It runs in five phases: an initial pause of a fixed 5 s at P_{start}, a ramp up to P_{top}, a hold of T_{pause} at P_{top}, a ramp down, and a final pause of a fixed 5 s at the end PEEP, after which the maneuver ends.



Figure 16. Operation view — a pressure-ramp maneuver rendered on the global scopes.⁷

Saline-bolus maneuver

The saline-bolus maneuver injects a contrast bolus into the perfusion model (per-region gamma-variate) to exercise the regional perfusion signal, and can be superimposed on an active ventilation mode. Both maneuvers are triggered from the Maneuvers view.

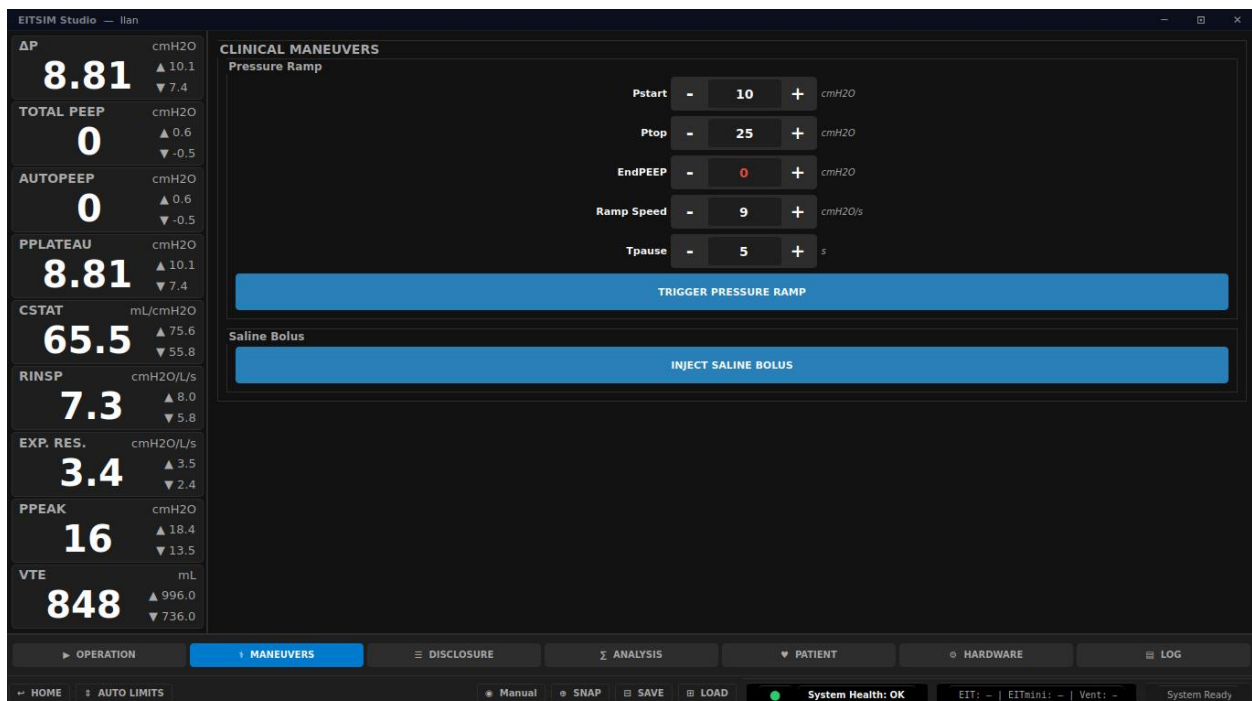


Figure 17. Maneuvers view — pressure-ramp and saline-bolus controls.

User Interface

Design language

The user interface deliberately imitates a bedside ICU monitor. It runs full-screen and frameless, with a custom title bar; a permanent left-hand numeric dashboard; a central area of sweeping waveform scopes; a bottom navigation bar; and a status bar carrying the system-health indicator. All colours and dimensions are defined centrally in a theme module rather than hard-coded, and are kept separate from the numeric constants.

Main window layout

The main window (MainWindow) is a frameless QMainWindow whose central area is a QStackedWidget of views selected from the bottom navigation bar. Its principal regions are:

- Title bar — a slim custom bar carrying the patient-name chip and window controls.
- Left numeric dashboard — a fixed-width column of nine metric tiles: ΔP (driving pressure), Total PEEP, AutoPEEP, Pplateau, Cstat (static compliance), Rinsp (inspiratory resistance), Exp. Res. (expiratory resistance), Ppeak and VTE. Each tile carries alarm limits and a tooltip, and is blanked to a dash while the simulation is idle.
- Central view stack — one view per navigation tab.
- Bottom navigation bar — seven mutually-exclusive buttons selecting the view.
- Status bar — HOME, AUTO LIMITS, a Manual (tooltip) toggle, a SNAP toggle, SAVE/LOAD of application state, and the system-health badge.

The seven views

The bottom navigation bar switches between seven views, each covered in detail in its own chapter where applicable:

Table 7. The seven navigation views.

View	Purpose
Operation	Real-time global waveforms (volume, flow, airway pressure) with the ventilation-mode selector and per-mode parameter staging.
Maneuvers	Pressure-ramp and saline-bolus triggers.
Disclosure	High-density regional scopes: per-compartment volume, flow and elastic pressure, plus ECG and regional hemodynamics.
Analysis	Static metrology: PEEP titration, P–V curves and the Costa method.
Patient	Demographics and the regional/global physiological model parameters.
Hardware	Serial-port configuration, the AM emulator controls, and the Hamilton C6 link console.
Log	The passive, colour-coded Event Log and the continuous-CSV recorder selector.



Figure 18. Operation view — snap-to-peak measurement cursors on the global scopes.

Regional disclosure

The Disclosure view presents the simulation's regional detail: per-compartment volume, flow and elastic-pressure scopes for the four lung regions, and separate scopes for the ECG/cardiac trace and the regional hemodynamic (pulsatility) signals. Its time axis is linked with the Operation scopes.



Figure 19. Disclosure view — regional volume, flow and alveolar-pressure scopes for the four lung compartments.



Figure 20. Disclosure view — ECG/cardiac trace and regional hemodynamic (pulsatility) signals.

Widget library

Two custom widgets carry most of the interaction and telemetry:

SweepBarCanvas

SweepBarCanvas is the medical sweep scope. It renders in the ICU eraser-bar style — a moving bar erases the oldest trace as the newest is drawn — which is tear-free while scrolling. Data is written into a NumPy ring buffer at 100 Hz and the redraw is throttled to roughly 20 frames per second, so no data is lost regardless of paint rate. It supports multiple traces, freezing and linking the time axis across scopes, snap-to-peak measurement cursors, a resizable time window, and buffer save/restore for state persistence.

NumericInput

NumericInput is a validated staging control laid out as label, minus/value/plus, and unit suffix. A value can be typed directly or stepped with press-and-hold acceleration on the plus/minus buttons. It follows a staging model: an edited-but-not-applied value is shown 'dirty' in yellow and is not sent to the engine until a global Apply, enforcing the safe-injection pattern; Discard reverts it. Hard minimum and maximum bounds are enforced per parameter.

Conventions

Two project-wide conventions are enforced in the interface. The regional colour convention (US37) is strict and universal: Anterior = Yellow, Posterior = Blue, Right = Green, Left = Red. The unit standard (US38) fixes the displayed units — volume in mL, pressure in cmH2O, flow in L/s, compliance in mL/cmH2O, resistance in cmH2O/(L/s) — permanently visible in axis labels and unit suffixes.

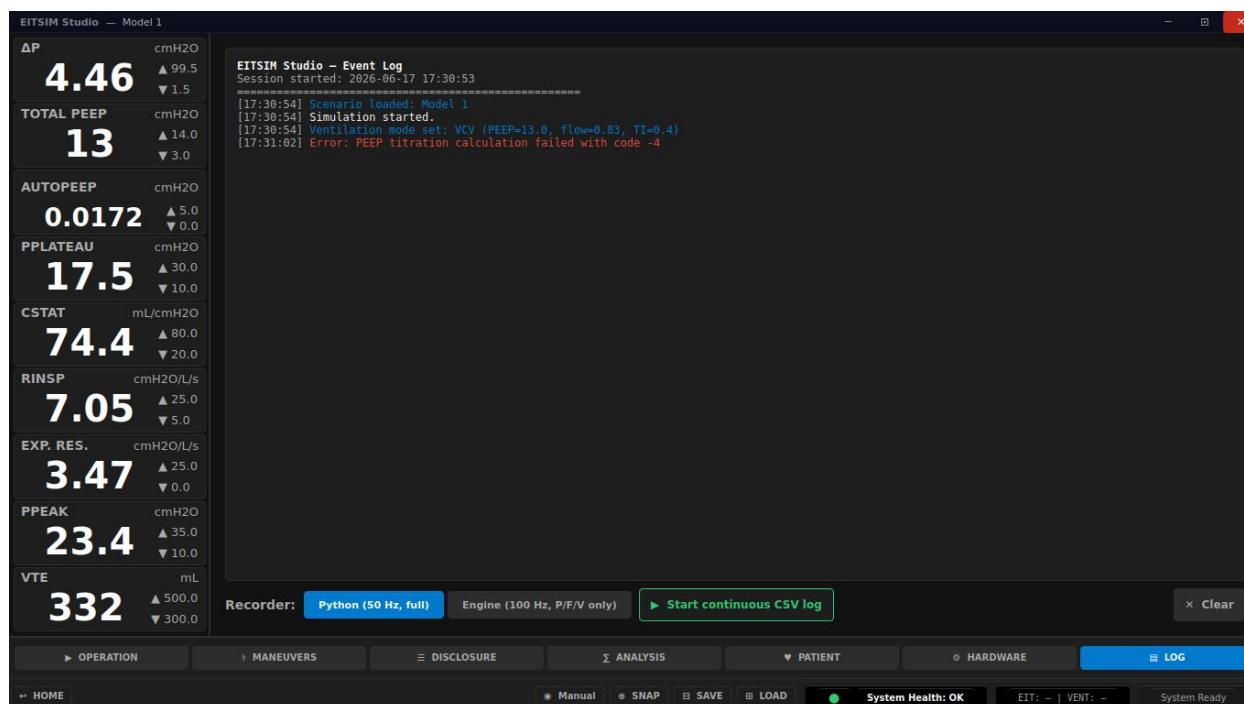


Figure 21. Event Log view — timestamped system events including engine model-error reporting.

WSLg full-screen note

On Wayland/WSLg full-screen, any chrome widget whose minimum width grows past the screen crashes the compositor. The interface therefore uses eliding labels with a zero minimum width (retaining the full size hint) for variable-width chrome, so text truncates gracefully instead of forcing the layout wider than the display.

Analysis and Metrology

Overview

The Analysis view provides static respiratory-mechanics metrology, computed by the engine rather than synthesised in the interface. It is organised into three panels selected from a tab strip: PEEP Titration, P–V Curves, and the Costa Method. Curves can be exported to a standardised CSV and a reference CSV can be overlaid for comparison. The result-structure fields cited here are defined in `EITSIM_core_for_python.md` and the analysis mathematics in the `pv_analysis` library documentation.

PEEP titration

The PEEP-titration panel sweeps PEEP across a range and reports both theoretical and analytical indices, differentiated by ventilation mode (VCV versus PCV). The engine call is `calculate_peep_titration(mode, value, p_min, p_max, p_step)`, where the mode selects PCV (with the driving pressure as value) or VCV (with the tidal volume as value). The panel plots secondary compliance, regional distribution, and the derived recruitment/overdistension indices against PEEP.

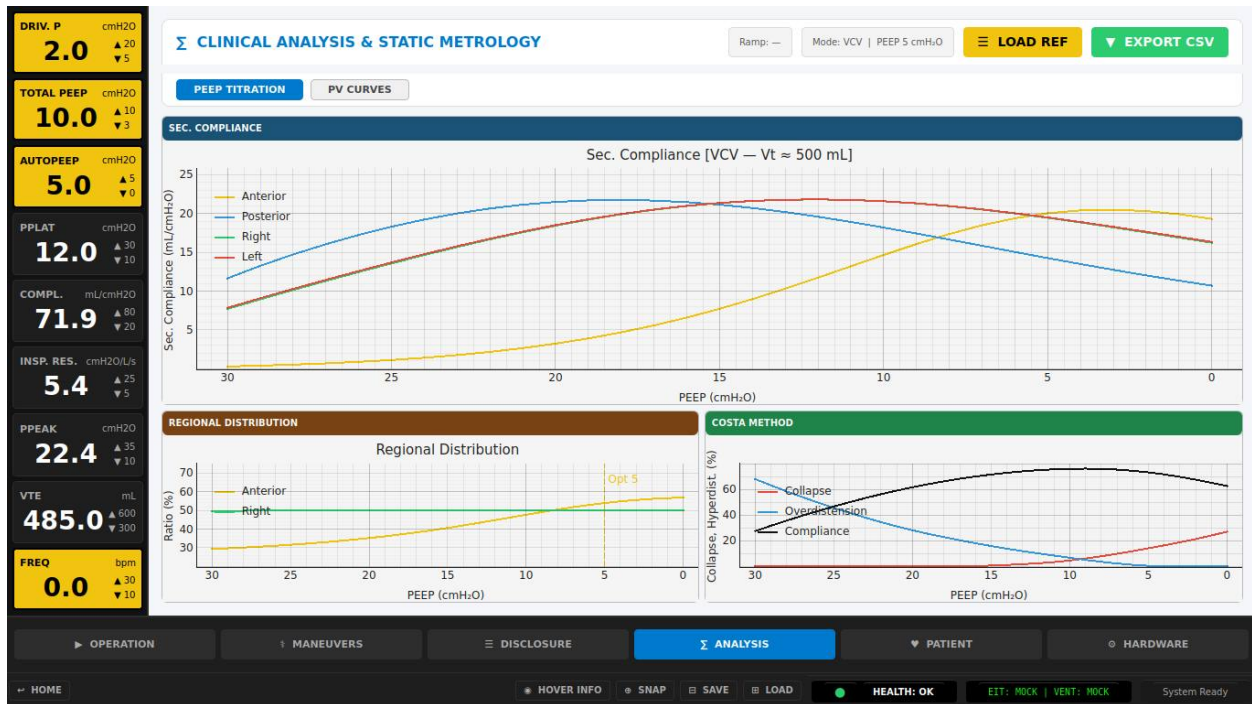


Figure 22. Analysis view — PEEP titration: secondary compliance, regional distribution and Costa-method curves.

P–V curves

The pressure–volume panel is always available because it depends only on the patient physiology, not on an active ventilation mode. It is computed by `calculate_full_pv_analysis(p_min, p_max, p_step)` and returns the per-compartment volume and compliance curves together with global compliance, the collapse percentage and the hyperdistension percentage. Measured pressure-ramp hysteresis loops from a ramp maneuver can also be displayed.

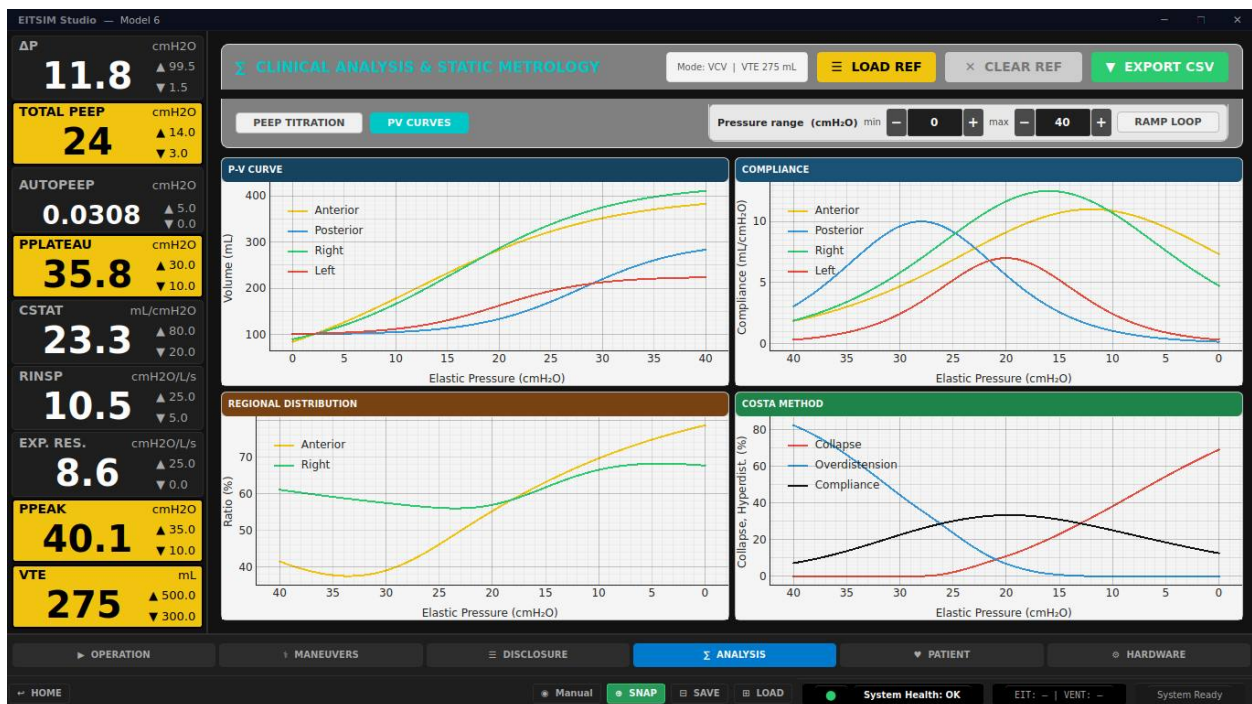


Figure 23. Analysis view — measured pressure-volume curve and Costa-method analysis.

Costa method

The Costa-method panel plots collapse, overdistension and compliance against elastic pressure, with the Anterior/Right regional ratio metrics. These indices quantify, per PEEP level, how much of the lung is collapsed versus overdistended — the basis for identifying an optimal PEEP.

Export and reference overlay

Each analysis can be exported to a standardised CSV whose filename encodes the mode and settings, and a previously-saved reference CSV can be overlaid on the live curves for comparison. CSV export is gated on completed breaths so that an analysis is not exported from an incomplete maneuver.

Guard against out-of-range sweeps

If a titration or P–V sweep requests a step whose tidal volume exceeds total lung capacity, the engine returns model error code -4 (Runge–Kutta failure). Rather than leaking the C++ diagnostic to the console, the Analysis view presents a guard banner on the affected panel explaining that the requested tidal volume exceeds total lung capacity and that the range should be reduced.

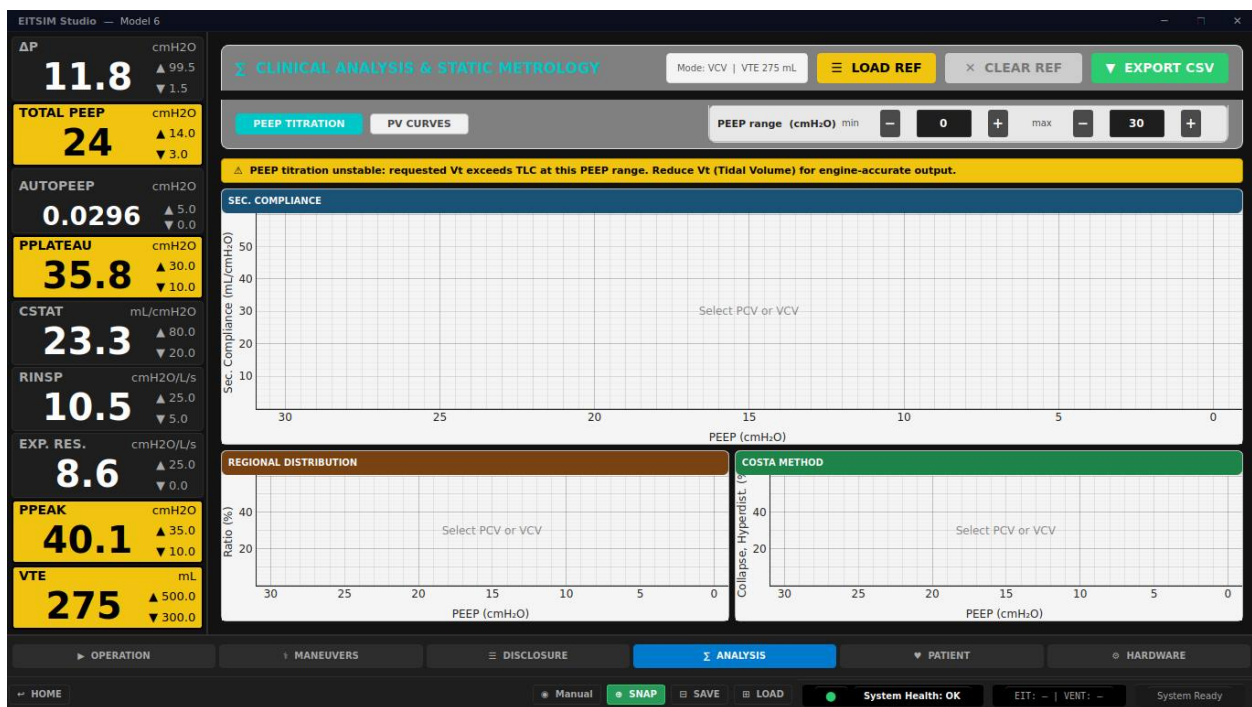


Figure 24. Analysis view — guard banner when a PEEP-titration step exceeds total lung capacity.

Live Measurements and System Health

Live respiratory-mechanics estimation

While the simulation runs, the engine's LSFestimator continuously fits the pressure/flow/volume waveforms to produce breath-by-breath respiratory-mechanics estimates. AppState polls these at 5 Hz (a 200 ms timer) via `get_latest_estimates()` and emits `metrics_updated` to the numeric dashboard. The estimator's `RespiratoryEstimates` structure carries the plateau pressure, AutoPEEP, static compliance, inspiratory and expiratory resistance, and the measured respiratory rate and I:E ratio. The all-zero warm-up window, before the estimator has seen a full breath, is suppressed so the dashboard does not flash spurious zeros at start-up.

Derived dashboard quantities

The left dashboard shows nine quantities, several of them derived from the raw estimates:

- ΔP (driving pressure) = $P_{\text{plateau}} - (PEEP_{\text{base}} + \text{AutoPEEP})$, where $PEEP_{\text{base}}$ is the Hamilton C6-measured PEEP when under Hardware-in-the-Loop control, otherwise the configured PEEP.
- Total PEEP = configured/measured PEEP + AutoPEEP.
- P_{peak} — the peak airway pressure locked at each completed breath.
- VTE — the expiratory tidal volume locked at each completed breath.
- P_{plateau} , Cstat, Rinsp, Exp. Res., AutoPEEP — reported directly from the estimator.

Breath-cycle detection

Two breath-cycle detectors coexist. The default detector is phase-transition based, keyed on the engine's ventilation phase, and is used with the internal ventilator. Under real Hamilton C6 Hardware-in-the-Loop control the reported ventilation-phase status bit flickers unreliably, so a second, volume-hysteresis based detector is used instead; it locks P_{peak} , VTE and AutoPEEP on a completed breath detected from the volume trace. Both return a completed-breath signal that gates the analysis exports.

System-health indicator

The status bar carries a system-health badge that merges two drivers: the EIT link drop-rate and the engine model-error code, with model errors taking priority. The link drop-rate is polled at 1 Hz via `get_eit_link_status()` and classified against fixed thresholds.

Table 8. System-health link-quality thresholds.

Drop rate	State	Colour	Meaning
< 0.1 %	OK	Green	Link healthy.
0.1 – 1 %	WARNING	Yellow	Degraded link; investigate.
> 1 %	CRITICAL	Red	Link unacceptable — invalidates a Hardware-in-the-Loop test.

A critical engine model error (for example a Runge–Kutta failure or a barotrauma/TLC-exceeded condition) overrides the link status and drives the badge to its critical state.

Alarm limits

Each dashboard tile (a `MetricDisplay`) shows its value with a maximum limit above and a minimum limit below. Tapping a tile opens an editor to adjust the alarm limits; when the value falls outside its limits the tile background turns yellow. An AUTO LIMITS action in the status bar sets sensible limits from the current values.

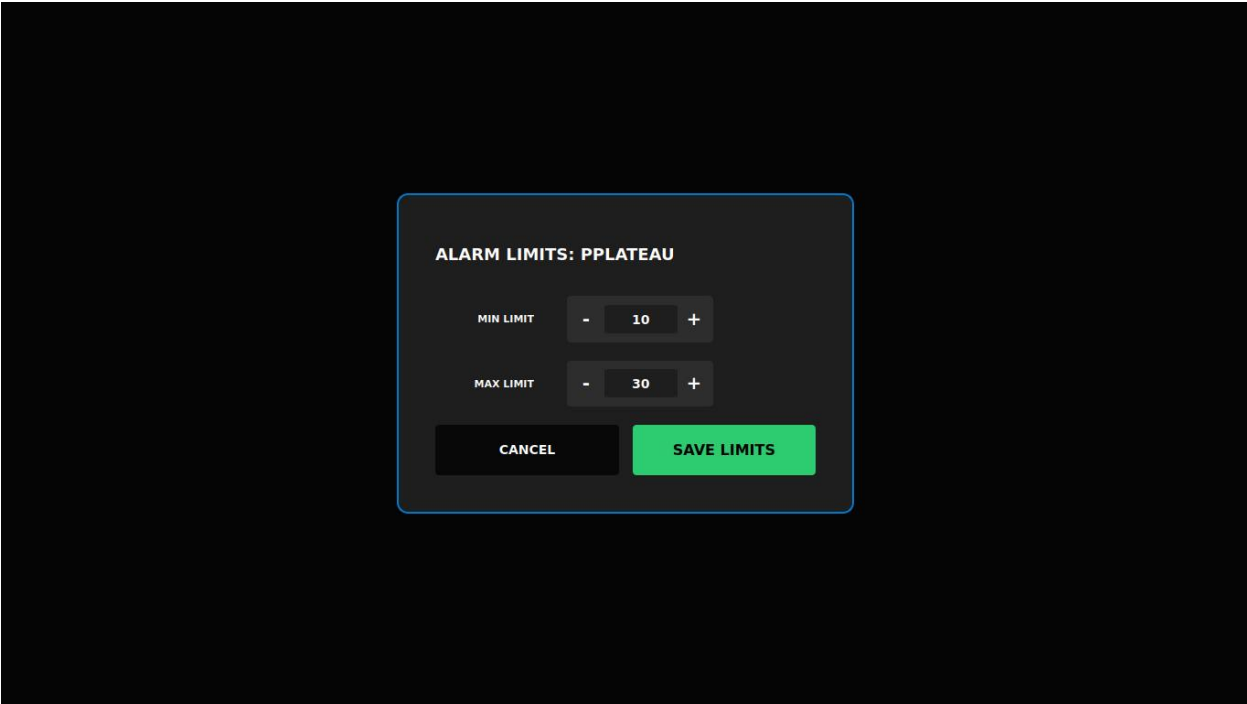


Figure 25. Editing an alarm limit on a dashboard metric.

Hardware Interfaces

Overview of the four interfaces

EITSIM Studio connects to external equipment through four hardware interfaces. Three are serial and one is TCP; three carry data out of the simulator and one carries live control into it. All four are routed through AppState facade methods, and their setup lives in the Hardware view. This section documents the two serial output devices; the Hamilton C6 and the AM/PIM emulator follow in their own sections.

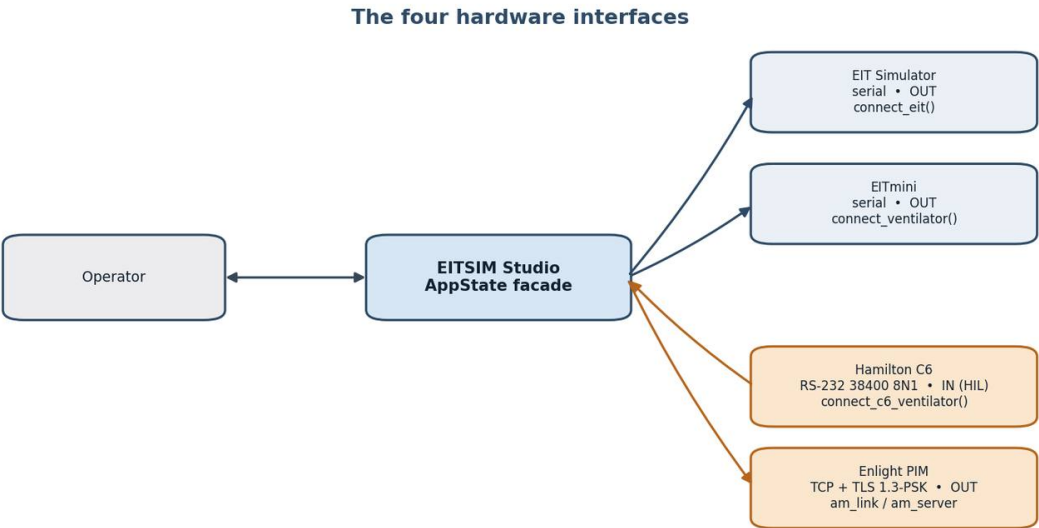


Figure 26. The four hardware interfaces and their data directions.

Table 9. The four hardware interfaces and their handling.

Interface	Direction	Transport	AppState call
EIT Simulator	Output	Serial	connect_eit(port, sample_rate, mode)
EITmini	Output	Serial	connect_ventilator(port)
Hamilton C6	Input (HIL)	Serial RS-232	connect_c6_ventilator(port)
AM / PIM emulator	Output	TCP + TLS 1.3-PSK	start_am_emulator(host, port_offset, belt_csv)

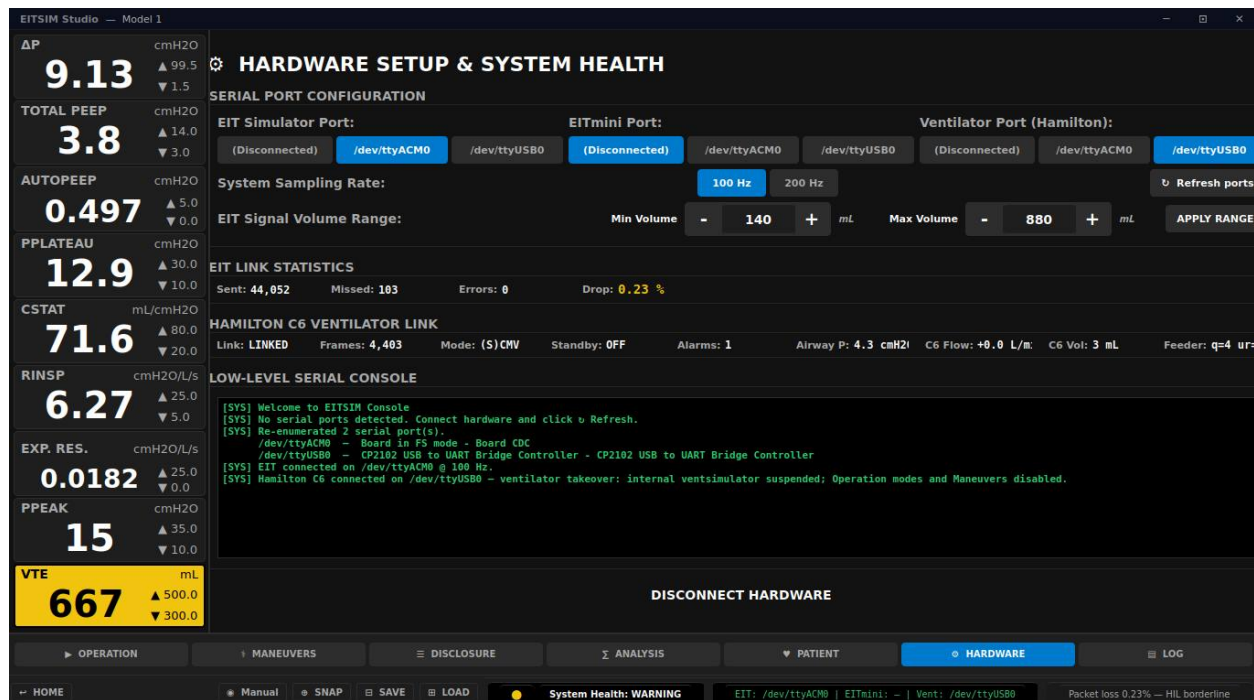


Figure 27. Hardware view — serial-port configuration, EIT link statistics and the Hamilton C6 link.

EIT Simulator

The EIT Simulator is a serial output device. It is connected with `connect_eit(port, sample_rate, mode)`; the application streams the engine's regional conductivity signal to it. When present, the EIT Simulator also provides a precise 100 Hz hardware clock that paces acquisition, which is why the profiled sample rate should sit close to 100 Hz. Link quality is reported by `get_eit_link_status()`, whose drop-rate feeds the system-health badge (see the Live Measurements chapter).

EITmini

The EITmini is a second serial output device, connected with `connect_ventilator(port)`. The engine method name is historical; in the interface the port is labelled as the ventilator/Hamilton port. It carries the ventilation data stream to the EITmini device.

The Hardware view

The Hardware view owns serial-port configuration for all interfaces. It offers real COM/TTY ports and MOCK ports for bench work without hardware, a configurable system sampling rate, and a low-level serial console for diagnostics. Connection changes are reported through `hardware_link_changed` and `link_health_changed`, which feed the status-bar health badge and the Event Log; a single disconnect action tears down all serial links at once. The view uses a reflowing layout so that port cells wrap rather than force the window wider than the display (the WSLg minimum-width constraint noted earlier).

Hamilton C6 Hardware-in-the-Loop

The Hamilton C6 interface turns EITSIM Studio into a digital twin under Hardware-in-the-Loop (HIL) control: a real Hamilton C6 ICU ventilator, connected to a mechanical test lung, streams its live airway pressure and flow into the simulator, which drives the modelled lung with them in place of the internal ventilator. The EIT device downstream therefore sees a regional impedance signal generated by the real ventilator's output rather than by a synthetic waveform. The design and decoding are specified in HIL_Action_Plan_Hamilton_C6.md.



Figure 28. The Hamilton C6 ventilator used as the HIL drive source.

Physical setup

The C6 exposes a female DB9 RS-232 port wired as data-terminal equipment (DTE); connecting it to the host therefore requires a crossover / null-modem cable. The ventilator's patient circuit is connected to a mechanical test lung, which provides the physical load the C6 controls.

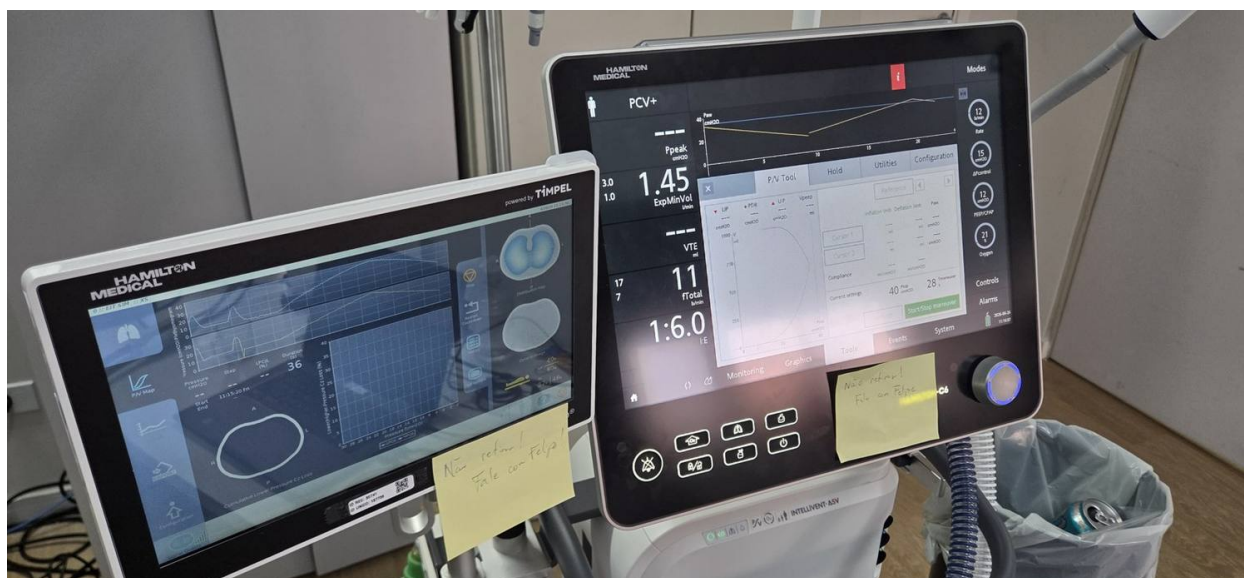


Figure 29. Hardware-in-the-loop bench — EITSIM Studio connected to both EITmini/Pulmomap and Hamilton C6.

Timing and rate conversion

The C6 sends a block every 100 ms carrying five samples at 20 ms resolution, i.e. an effective 50 Hz. The engine runs at 100 Hz, so each C6 sample is duplicated (a zero-order hold) to upsample 50 Hz to 100 Hz. The C6 reports volume in mL and flow in mL/s; these are divided by 1000 to litres before being handed to the model.

Engine integration

Under HIL the internal ventilator-signal generator is suspended and the C6's airway pressure/flow is injected as the ventilation control intent. The drive carries a control variable so the model integrates correctly per mode: volume-targeted modes (VCV, CMV, SIMV) drive the lung by inspiratory flow with a clamped expiratory pressure, while pressure-targeted modes (PCV, APVsimv) drive by airway pressure. The global pressure/flow/volume pass through to the regional map.

Metadata

Beyond the waveforms, the C6 stream carries metadata that the application extracts and uses:

- Breath number — used as the temporal synchronisation anchor.
- Ventilation mode — displayed and used to select the drive control variable and to label the analysis.
- Standby — when set, freezes the simulation.
- Active alarms — surfaced in the interface.

Takeover behaviour and the operator experience

Connecting the C6 with `connect_c6_ventilator(port)` marks the C6 active, switches breath detection to the volume-hysteresis detector, and auto-starts the engine so the simulator can run with only the C6 plugged in. A dimming overlay is drawn over the Operation and Maneuvers controls to make clear they are inert while the ventilator is in control.

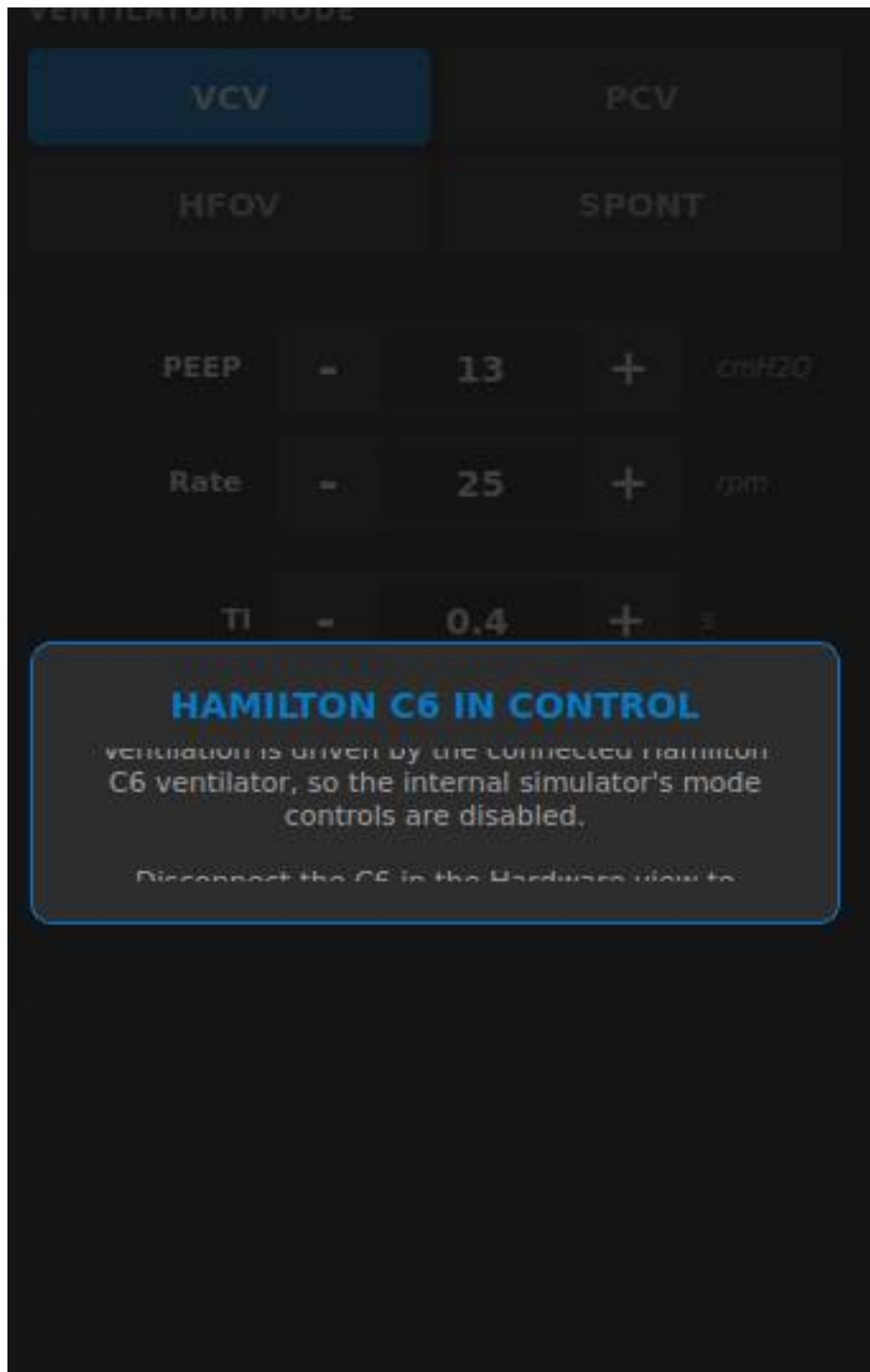


Figure 31. 'Hamilton C6 in Control' overlay — Operation and Maneuvers controls are inert under HIL.

The live waveforms in the Operation view are then driven by the C6, and the analysis view follows the live C6 mode (for example a PEEP titration labelled with the C6's current mode). The C6-reported expiratory tidal volume (VTE) is displayed as the reference against which the simulated VTE is validated.



Figure 32. Operation view driven live by a connected Hamilton C6 (HIL).

The Hardware view shows a Hamilton C6 link console with the live link state, the received frame count, the current mode, standby and alarm status, the airway pressure, and the internal feeder status. This console is the primary diagnostic for the HIL link.

Figure 33. Hamilton C6 ventilator-link console — link state, frame count, mode, airway pressure and feeder status.

Practical notes

The C6 receive-link API and volume-mode drive are part of the current 40-method eitsim_core build (connect_c6_ventilator, get_c6_status, disconnect_c6_ventilator). On the host side, FTDI USB-serial adapters are strongly preferred; CH340-based adapters and usbipd USB-forwarding proved unstable during bench bring-up.

AM / PIM emulator

The AM/PIM emulator is the most significant capability delivered during this development cycle. EITSIM Studio emulates the EIT Acquisition Module (AM) so that a real, unmodified Enlight PIM (Processing & Interface Module) performs live EIT monitoring driven entirely by the simulated physics — with no phantom and no physical AM box. The device-side protocol is specified in [internal ref]; the summary here documents what the emulator implements and how it is integrated, and refers the reader to that document and to AM_HANDOFF.md for the full detail.

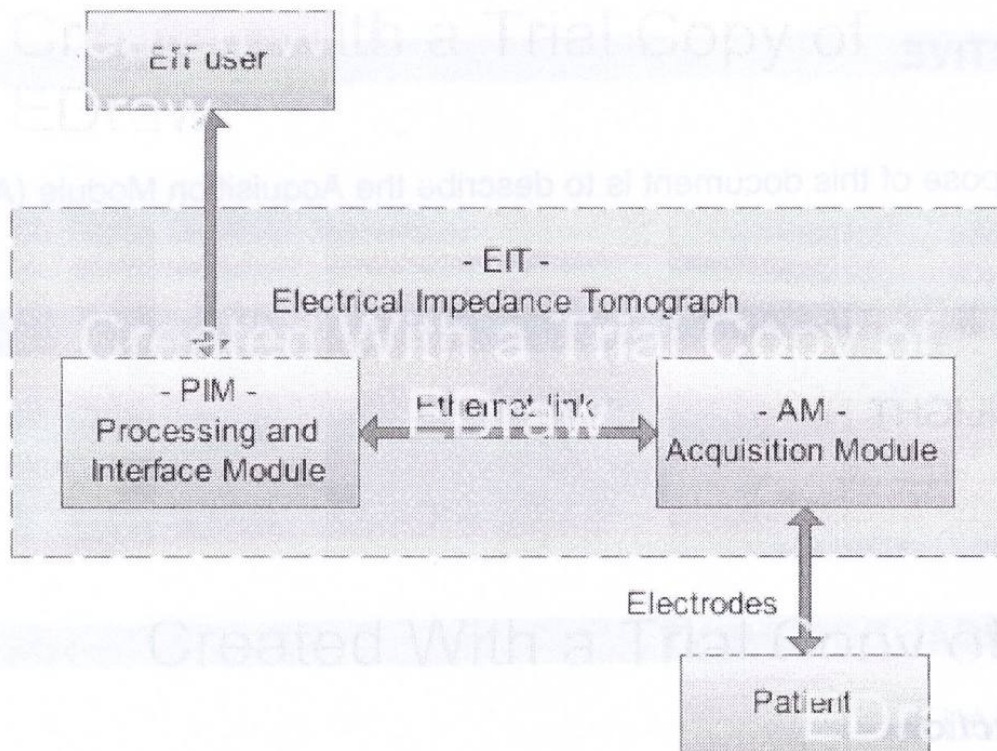


Figure 1. EIT modules: AM and PIM.

Figure 34. EIT device architecture: the Acquisition Module (AM) and the Processing & Interface Module (PIM). (Source: [internal ref].)

The reduced 4-region forward model

The EIT data the emulator streams is generated by a reduced four-region forward model implemented in `forward_model.py`. The model is a first-order linearisation about a baseline: the 1024-measurement boundary-voltage vector is $V = V_0 + J_{\rho} \cdot \Delta\rho$, where V_0 is the baseline vector and J_{ρ} is a 1024×4 reduced Jacobian mapping the four regional resistivity changes (Anterior, Posterior, Right, Left) to boundary voltages. The reduced Jacobian is obtained by collapsing the thousands of finite-element pixels of a full Jacobian into the four anatomical zones.

JACOBIAN & LINEARIZATION

Sensitivity Matrix

The Jacobian (J) maps internal conductivity changes to boundary voltage changes. It is calculated using a **Finite Element Method (FEM)** discretization of the space.

FORWARD PROBLEM:

$$\Delta V = J \cdot \Delta \rho$$

Inverting it step-by-step

$$J^T \cdot \Delta V = (J^T \cdot J) \cdot \Delta \rho$$

$$(J^T \cdot J + \lambda^2 R)^{-1} \cdot J^T \cdot \Delta V = (J^T \cdot J + \lambda^2 R)^{-1} \cdot (J^T \cdot J) \cdot \Delta \rho$$

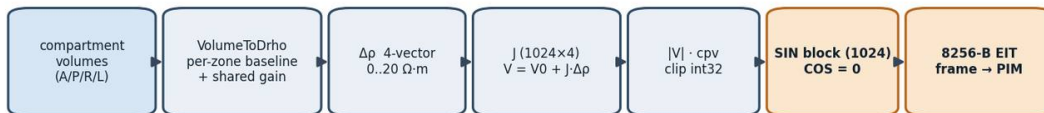
INVERSE PROBLEM (with Regularization):

$$\Delta \rho = (J^T \cdot J + \lambda^2 R)^{-1} \cdot J^T \cdot \Delta V$$

R is known as the **penalty term** (or regularization matrix)

Figure 40. Forward-model concept — the Jacobian maps regional resistivity change ($\Delta \rho$) to boundary-voltage change ($V = V_0 + J \cdot \Delta \rho$).

Reduced 4-region forward model



linearized about baseline · purely resistive medium · gain auto-fit so $\max|V_0| \approx 1.5e9$ counts

Figure 41. Reduced 4-region forward-model pipeline: compartment volumes -> $\Delta \rho$ -> J -> boundary voltages -> wire counts (SIN block).

Because the simulated medium is purely resistive, only the in-phase (SIN) block is populated and the quadrature (COS) block is zero; the transmitted value is $|V|$. Wire counts are computed as $\text{clip_int32}(\text{round}(|V| \cdot \text{cpv}))$, where cpv (counts per volt) is auto-fitted so that the peak baseline magnitude lands near the empirically-proven $\sim 1.5 \times 10^9$ count peak, avoiding int32 overflow. A VolumeToDrho mapper converts the four compartment volumes to $\Delta \rho$ in the range 0–20 $\Omega \cdot \text{m}$: each zone rises from its own end-expiratory baseline (the minimum seen), and a single shared gain (from the largest per-zone tidal excursion) scales all zones, preserving the inter-zone contrast so the reconstructed image is genuinely regional rather than a uniform pulse.

Embedding in EITSIM Studio

The emulator runs in-process inside EITSIM Studio, controlled by `src/core/am_link.py` (AMEmulatorController). There is no subprocess and no second engine: the controller places the emulator on the import path, loads the reduced-Jacobian forward model and a captured analog header (so the PIM's electrode-contact gate passes), and installs a source closure that reads the latest simulated regional volume and updates the mapper. The live volume is published from the engine callback by the lock-free single-writer rebind described in the architecture chapter, so the emulator's socket thread never races the engine. The AM emulator controls live in the Hardware view.

Data Recording and Persistence

Continuous CSV logging

EITSIM Studio can record continuous raw data to CSV, selectable from the Log view. Two backends are offered, with different rates and coverage; the full column dictionaries are given in Appendix C.

Table 10. The two CSV recorder backends.

Backend	Rate	Columns	Coverage
Python recorder (default)	50 Hz	34	Global P/F/V, regional volumes and flows, regional alveolar pressures, ECG, five hemodynamic channels, and an 11-column Hamilton C6 HIL block (including C6_VTE_mL, the correct VTE reference).
Engine recorder	100 Hz	13	Global signals only, written off the GIL; no ECG, no hemodynamics.

In both formats the first six rows are `#` comment lines carrying session metadata, and the default filename follows the pattern `EITSIM_Log_YYYYMMDD_HHMMSS.csv`. The Python backend is the default because it captures the regional, ECG and hemodynamic detail and the Hamilton C6 HIL block; the engine backend trades that coverage for a higher rate and lower overhead.

The plot_log viewer

A built-in viewer, `plot_log.py`, renders a recorded CSV as a four-row dark dashboard. Run from the virtual environment, it auto-finds the most recent log or opens a file picker, and it uses the same regional colour convention as the application.

Application-state save and load

The complete application state can be saved to and loaded from a `.eitsim` file (JSON). The serialisation captures the Operation, Maneuvers, Patient, Hardware and Analysis settings, the waveform state (eight frozen scope buffers), and the interface state. A loaded state lands stopped, so the operator explicitly re-starts the simulation. Independently, session state — the last COM port and the active view — is auto-saved to a single settings file in the user's home directory on close, so the application reopens where it was left.

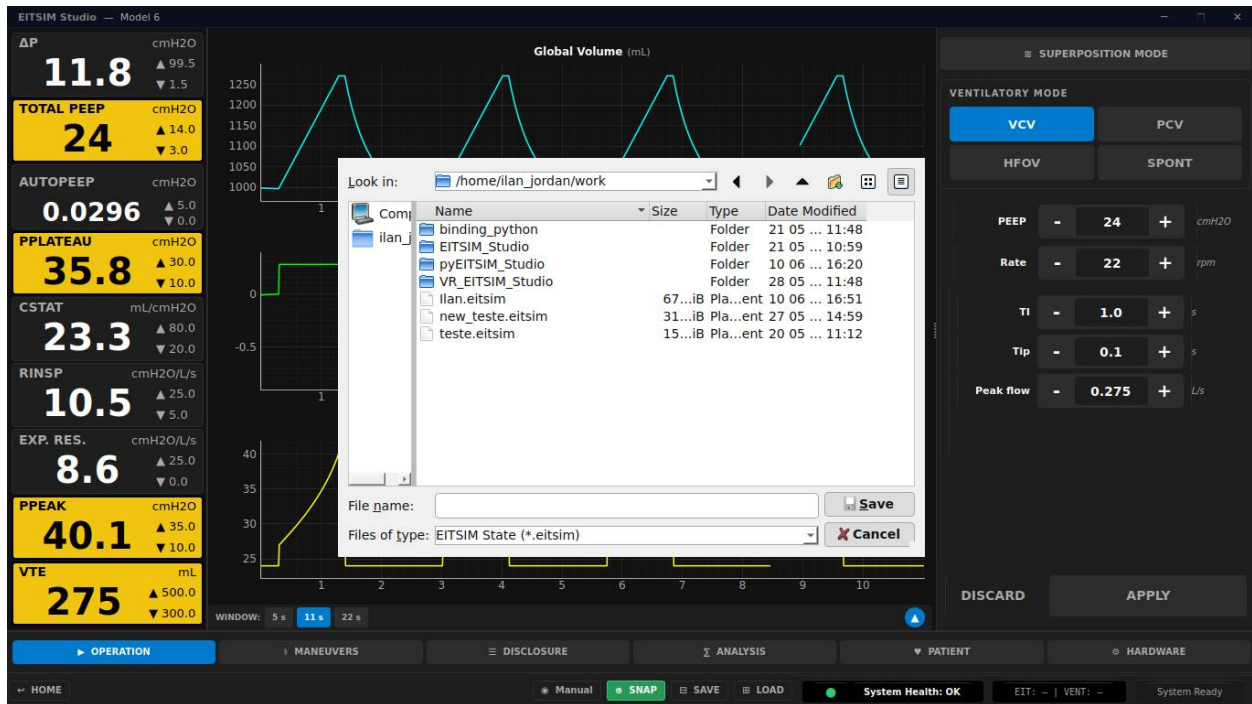


Figure 45. Saving the full application state to a .eitsim file.

Event Log

The Log view also hosts a passive, colour-coded, timestamped Event Log. It records lifecycle and hardware events — simulation start/stop, hardware connect/disconnect and link-health changes, engine model-error transitions, and analysis failures — providing an audit trail for a test session alongside the recorder selector.

Build, Deployment and Runtime

Runtime environment

The reference runtime is the `eitsim_env` virtual environment on CPython 3.12. The application is launched through `run.sh` on Linux/WSLg or `run.bat` on Windows rather than by invoking the Python entry point directly; the launchers activate the virtual environment and route the bytecode cache to a project-local directory.

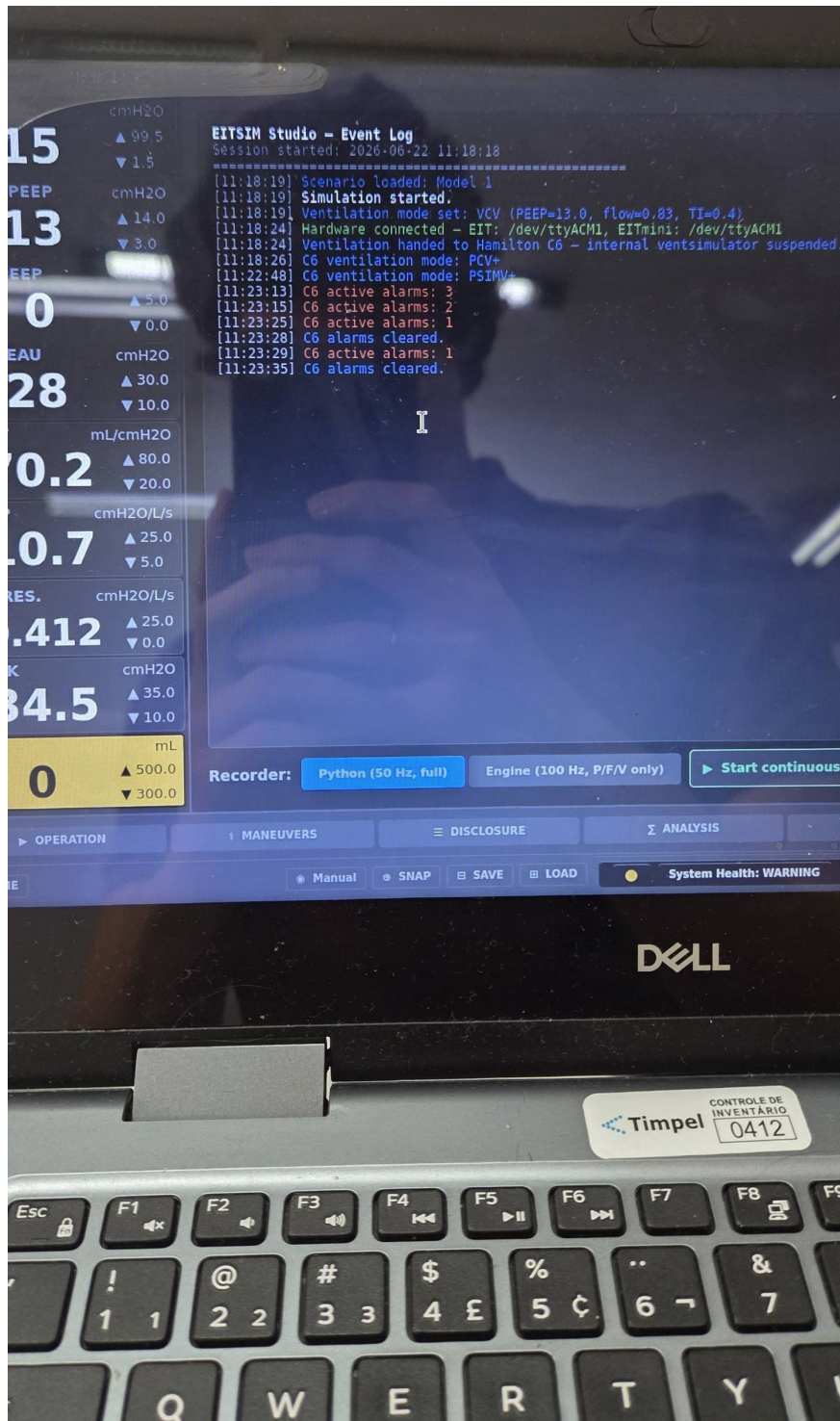


Figure 46. EITSIM Studio running on a Windows laptop.

On Linux/WSLg the launcher also exports the Qt/Wayland environment variables the application needs: `QT_SCALE_FACTOR=1` and `QT_AUTO_SCREEN_SCALE_FACTOR=0` (so display scaling does not inflate the Wayland buffer past the compositor size) and `QT_WAYLAND_DISABLE_WINDOWDECORATION=1` (so client-side decoration shadows do not inflate the surface buffer and crash the WSLg compositor).

Prebuilt engine binaries and ABI selection

The engine is distributed as prebuilt per-ABI binaries; the application selects the matching one at import time. `main.py` places `src/core` and then `src/Library_cpp` on the import path and lets CPython match the ABI tag of the running interpreter.

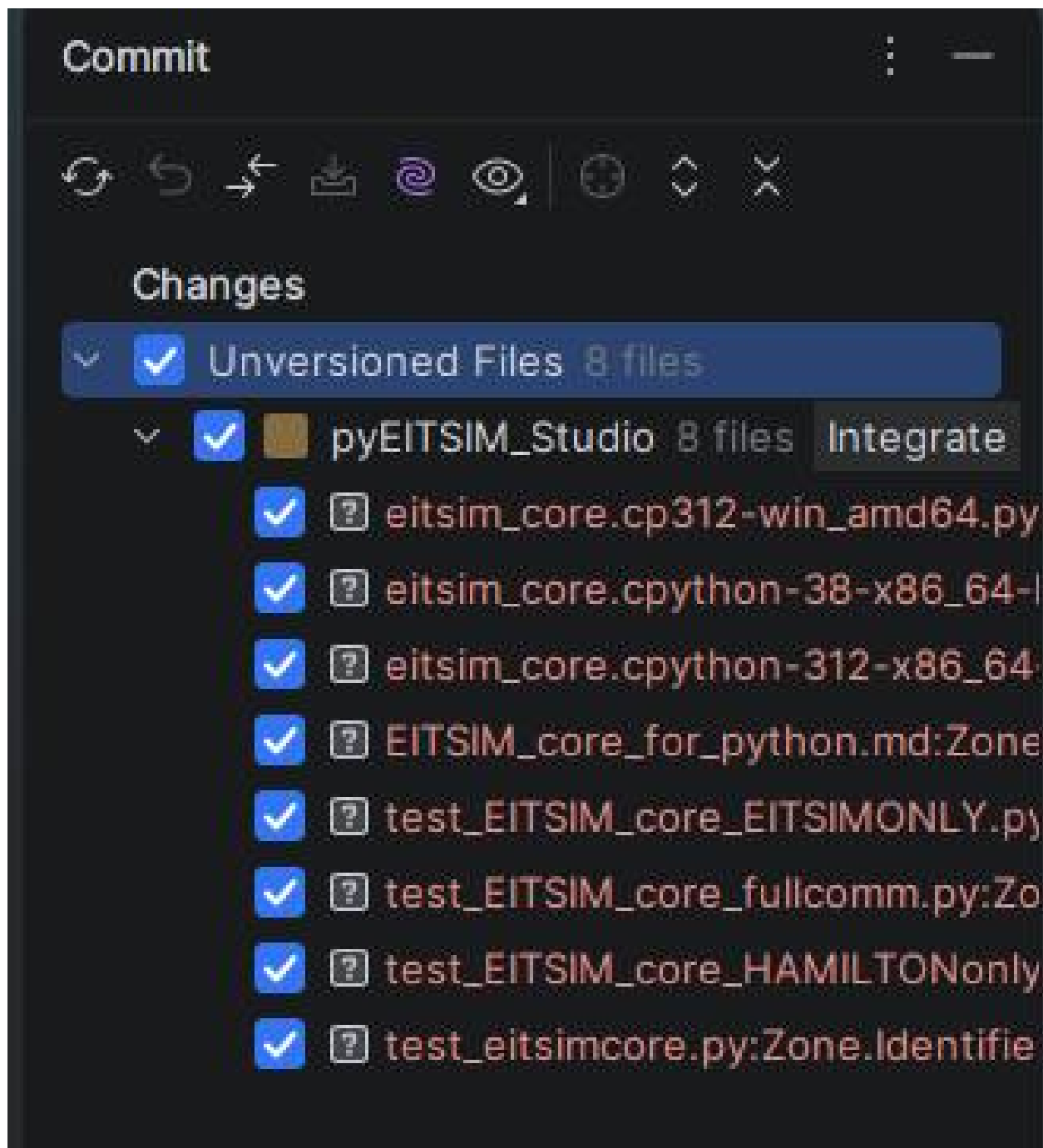


Figure 47. Per-ABI prebuilt eitsim_core engine binaries shipped in the repository (cp38/cp311/cp312 Linux, cp312 Windows).

Table 11. Prebuilt engine binaries.

Platform	Python	Binary
Linux (Ubuntu 20.04)	3.8	eitsim_core.cpython-38-x86_64-linux-gnu.so
Linux	3.11	eitsim_core.cpython-311-x86_64-linux-gnu.s o
Linux	3.12	eitsim_core.cpython-312-x86_64-linux-gnu.s o
Windows	3.12	eitsim_core.cp312-win_amd64.pyd (MinGW-w64)

Cross-platform matrix and dependencies

The modern dependency stack (in requirements.txt) is `numpy ≥ 2.4`, `pyqtgraph ≥ 0.14`, `PySide6 ≥ 6.11` with `shiboken6`, `pyserial`, and `PyOpenGL`. On Ubuntu 20.04 (glibc 2.31) the pins step back to `PySide6 == 6.4.3` / `shiboken6 == 6.4.3` on Python 3.8 or 3.11. On Windows, Python 3.12 is required to load the `cp312` binary.

Table 12. Supported platforms.

Platform	Python	Notes
Ubuntu 24.04	3.12	Primary development environment (modern stack).
Ubuntu 20.04	3.8 / 3.11	Legacy stack, PySide6 6.4.3.
Windows 10/11	3.12	cp312 binary; run.bat launcher.

Rebuilding the engine

The authoritative engine source is a separate repository (branch C6) that builds the full 40-method engine. The `src/eitsim_studio` tree vendored in the application repository is a stale pre-C6 snapshot and must not be built from. A correct build exposes 40 `SimulationFacade` methods including `connect_c6_ventilator`.

Each binary can only be built on its own platform, and a rebuilt binary is dropped into both `src/core` and `src/Library_cpp`. The Windows `.pyd` is a MinGW-w64 build (not MSVC — the engine's CMake is a GNU/MinGW build), produced on Windows via `build_engine_win.bat` with the MinGW/Boost toolchain under a space-free path; it is not cross-buildable from Linux.

Verification and Validation

Approach

Verification and validation of EITSIM Studio rests on four pillars: validation of the physiological model against real patient data, bench validation of the Hamilton C6 Hardware-in-the-Loop path, end-to-end bring-up of the AM/PIM emulator against a real device, and a set of automated self-checks. There is no linter or continuous-integration pipeline; verification is performed manually against these criteria.

Model validation

The built-in scenarios were tuned and validated against real, anonymised PEEP-titration patient data used as ground truth. The comparison covered global compliance versus PEEP, the Anterior/Right regional ratios versus PEEP, and the Costa collapse and hyperdistension curves versus PEEP. This anchors the simulated regional mechanics to clinically-observed behaviour.

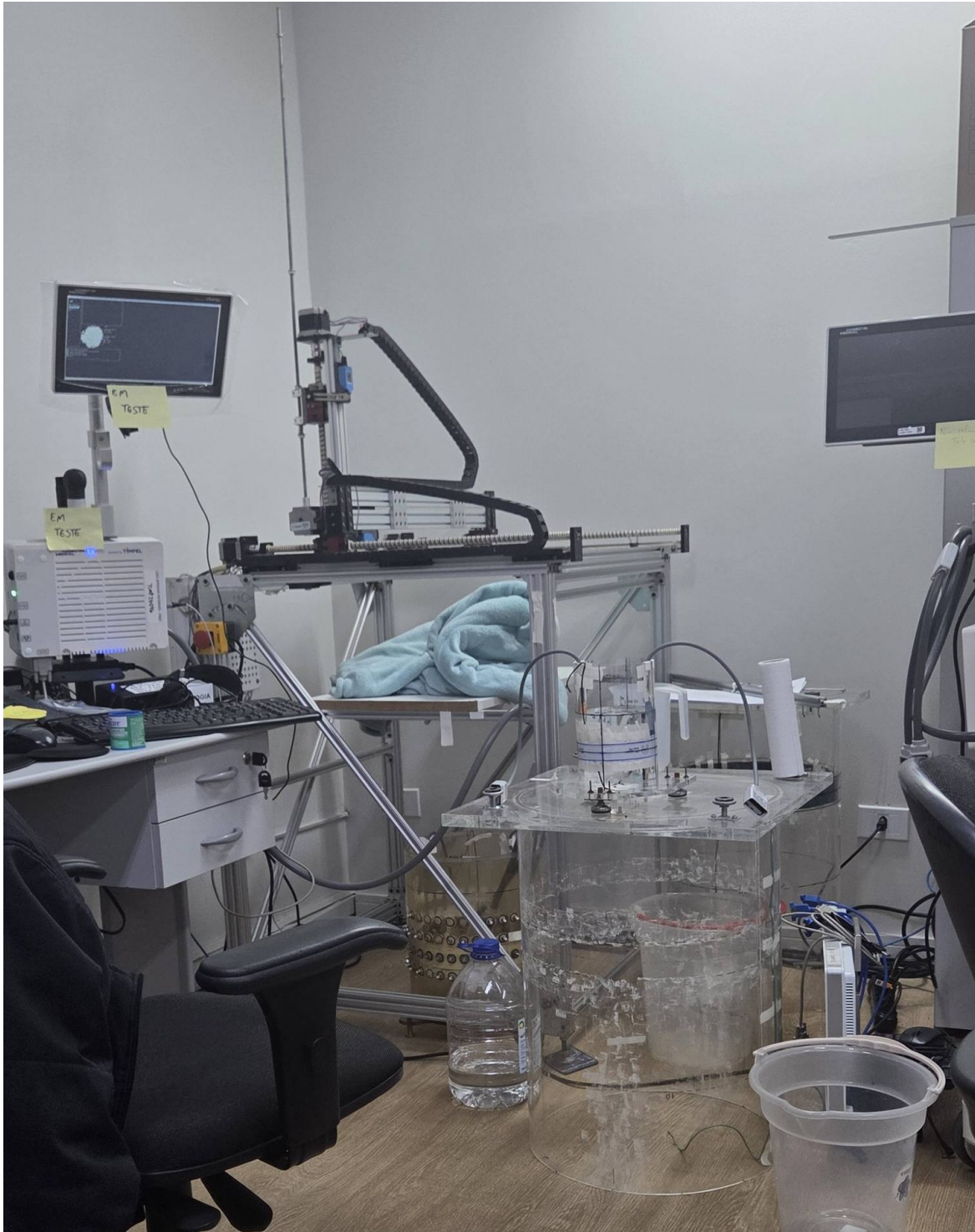


Figure 48. Bench validation setup — mechanical test lung and saline tank with the EIT belt.

AM/PIM live bring-up

The AM/PIM emulator was validated end-to-end against a real, unmodified Enlight PIM. The PIM completed every Start-Up gate — the TLS 1.3-PSK handshake (identity '[redacted]'), the XS belt calibration, the 32-electrodes-plus-reference check, the signal-quality check, the sensor-belt check, the minute-volume check (served via a fake Hamilton), and the [redacted] automatic current adjustment — and reached Launch, then rendered live regional EIT driven entirely by the emulated AM. No phantom and no physical AM box were used.



Figure 49. The complete test bench — EITSIM Studio connected to the PIM.

Link-quality gate

For any Hardware-in-the-Loop test the EIT link drop-rate is the acceptance gate: below 0.1 % is acceptable (OK), 0.1–1 % is a warning, and above 1 % is critical and invalidates the test. The system-health badge enforces this classification live.

Automated self-checks

Several automated checks guard the most error-prone components:

- `forward_model.py` carries a `__main__` self-check that exercises the reduced-Jacobian forward model.
- The AM emulator has a hardware-free self-test (`mock_pim.py`) that exits 0 on PASS, verifying that all eight sockets bind, that the [redacted] reply is a valid [redacted]-byte frame, that the EIT stream runs at ~50 frames/s with valid checksums, that ECG runs at ~50/s and ventilation at ~100/s, and that at least one monitoring packet is delivered.
- Engine smoke-test clients live in `src/Library_cpp/test_*.py` and `src/eitsim_studio/test/`.

Routine manual verification of a change consists of byte-compiling the edited Python, constructing the interface off-screen, and running the relevant self-checks before claiming the change works.

References

External literature

- Dormand, J. R. & Prince, P. J. 'A family of embedded Runge–Kutta formulae.' *Journal of Computational and Applied Mathematics* 6(1), 1980.
- Venegas et al. — sigmoidal pressure–volume model of the lung.
- Costa et al. — EIT-based estimation of alveolar collapse and overdistension.
- Rohrer — the parabolic airway-resistance equation.

Appendix A - Glossary and Abbreviations

This appendix defines the abbreviations and terms used in this document, in the EITSIM Studio context.

Table 14. Glossary and abbreviations.

Term	Definition
ABI	Application Binary Interface — the compiled-binary compatibility contract; each engine binary matches one Python ABI tag.
AM	Acquisition Module — the EIT front-end that acquires raw impedance data and serves it to the PIM.
APVsimv	Adaptive Pressure Ventilation (SIMV) — a pressure-targeted Hamilton mode.
AutoPEEP	Intrinsic positive end-expiratory pressure trapped in the lung beyond the set PEEP.
CMV	Continuous Mandatory Ventilation.
Cmax	Maximum compliance of a lung compartment.
Cstat	Static respiratory-system compliance.
CSV	Comma-Separated Values — the raw-data log format.
Dopri5 / RK	Runge–Kutta Dormand–Prince 5th-order adaptive ODE solver.
ΔP	Driving pressure = plateau pressure – total PEEP.
$\Delta\rho$ (delta-rho)	Regional resistivity change driving the forward model.
ECG	Electrocardiogram — here a trapezoidal trigger train.
EIT	Electrical Impedance Tomography — thoracic impedance imaging.
ETT	Endotracheal tube.
FEM	Finite Element Method — used to precompute the full Jacobian.
FloTrak	The OEM ventilation-data module referenced by the AM protocol.
HFOV	High-Frequency Oscillatory Ventilation.
HIL	Hardware-in-the-Loop — a real device in the simulation loop.
HRV	Heart-Rate Variability.
Jacobian	Sensitivity matrix mapping resistivity change to boundary voltage; here reduced to 1024×4 .
LSF / LSFestimator	Least-Squares Fit estimator of respiratory mechanics.
NL_MC	Non-Linear Multi-Compartment lung model.
ODE	Ordinary Differential Equation.
PCV	Pressure-Controlled Ventilation.
PEEP	Positive End-Expiratory Pressure.
PIM	Processing & Interface Module — the EIT device that processes AM data and drives the display.
PmaxC	Pressure of maximum compliance (sigmoid inflection point).
PSK	Pre-Shared Key — the TLS authentication secret (identity '[redacted]'), handled only by the tunnel.
pybind11	The C++/Python binding library exposing eitsim_core.
PV	Pressure–Volume (curve/analysis).
Qt / PySide6	The Qt 6 GUI toolkit and its Python bindings.
pyqtgraph	The real-time plotting library used for the scopes.
Rinsp / Rexp	Inspiratory / expiratory airway resistance.
SIMV	Synchronised Intermittent Mandatory Ventilation.
SIN / COS block	The in-phase and in-quadrature halves of the EIT data frame.
stunnel	The TLS terminator that handles the TLS 1.3-PSK tunnel on Windows.
TLC	Total Lung Capacity.
TLS	Transport Layer Security.
TOMS 748	The root-finding algorithm used for junction-pressure solving.
VCV	Volume-Controlled Ventilation.
VR	Residual Volume of a lung compartment.
VTE	Expiratory Tidal Volume.
VTLC	Compartment total-lung-capacity volume.
Vt	Tidal Volume.

Term	Definition
WSLg	Windows Subsystem for Linux GUI (Wayland-based).
XS	The EIT sensor-belt calibration/conversion (belt) constants.
ZOH	Zero-Order Hold — sample duplication used to upsample 50 Hz to 100 Hz.

Appendix C - CSV Column Dictionaries

This appendix reproduces the two CSV logging schemas. In both, the first six rows are comment lines prefixed with # (session metadata) that any parser can strip, and the default filename is EITSIM_Log_YYYYMMDD_HHMMSS.csv.

Python recorder — 50 Hz, 34 columns (default)

The Python recorder captures the full stream. The final eleven columns are Hamilton C6 Hardware-in-the-Loop context, populated only during a C6 session and blank/nan otherwise.

Table 17. Python recorder CSV columns (50 Hz, 34 columns).

Column	Description	Unit
time_s	Elapsed time from logging start	s
frame	Frame index	—
P_cmH2O	Global airway pressure	cmH2O
F_Ls	Global flow	L/s
V_mL	Global volume	mL
V_ANT/POST/RIGHT/LEFT_mL	Regional volumes (4 compartments)	mL
F_ANT/POST/RIGHT/LEFT_Ls	Regional flows (4 compartments)	L/s
P_ANT/POST/RIGHT/LEFT_cmH2O	Regional alveolar pressures (4 compartments)	cmH2O
ECG_mV	ECG signal	mV
Hemo_ANT/POST/RIGHT/LEFT/HEART_mL	Regional hemodynamic volumes (5 regions)	mL
C6_breath	Per-breath sync anchor (aligns rows to EIT device frames)	—
C6_mode	Reported C6 ventilation mode	—
C6_standby	C6 standby flag	—
C6_alarms	C6 active-alarm count	—
C6_pressure_cmH2O	Decoded C6 airway pressure	cmH2O
C6_flow_Lmin	Decoded C6 flow	L/min
C6_volume_mL	Decoded C6 volume	mL
C6_VTE_mL	C6-reported expired tidal volume (the correct VTE reference)	mL
C6_qdepth	Feeder queue depth (link health)	—
C6_underruns	Feeder underruns (link health)	—
C6_stalled	Feeder stalled flag (link health)	—

Counting the expanded regional and hemodynamic groups, the schema totals 34 columns: two index columns, three global signals, four regional volumes, four regional flows, four regional alveolar pressures, the ECG, five hemodynamic channels, and the eleven-column Hamilton C6 block.

Engine recorder — 100 Hz, 13 columns

The engine recorder is written from the C++ simulation thread without the GIL and is useful for timing/jitter characterisation. It captures global and regional pressure/flow/volume plus the mean alveolar pressure, with no ECG and no hemodynamics.

Table 18. Engine recorder CSV columns (100 Hz, 13 columns).

Column	Description	Unit
time_s	Elapsed time	s
P_cmH2O	Global airway pressure	cmH2O
F_Ls	Global flow	L/s
V_mL	Global volume	mL
V_ANT/POST/RIGHT/LEFT_mL	Regional volumes (4 compartments)	mL
F_ANT/POST/RIGHT/LEFT_Ls	Regional flows (4 compartments)	L/s
P_alv_mean_cmH2O	Mean alveolar pressure	cmH2O